SWE EXCELLENCE · FIELD GUIDE

STATE FIRST

A repeatable method for understanding, fixing, and extending code you did not write — and getting a result without guessing.

1 **Run it**
2 **Understand it**
3 **State it**
4 **Probe it**
5 **Pin it**
6 **Prove it**
7 **Log it**

Conditional breakpoints, watchpoints, and the thinking that makes them work.
Examples in C++ with gdb and lldb. Printed for use at a desk, with a pen.

Contents

Read this first 3

PART 1 — THE IDEA

1 The tool is only as good as the question 6
2 What the best engineers agree on 9
3 The seven steps 13

PART 2 — UNDERSTAND

4 Step 1 · Run it 17
5 Step 2 · Understand it 21

PART 3 — STATE IT

6 Step 3 · State it 27
7 From English to conditions 32
8 When you do not know the names yet 35

PART 4 — PROBE IT

9 Debugging a loop that never ends 39
10 Breakpoints and conditions 44
11 Watchpoints 49
12 Seeing state the way your editor shows it 53
13 Rewind, snapshots, and crashes 56
14 Make probes repeatable 59
15 Step 5 · Pin it 62

PART 5 — THE THREE FAMILIES

16 Intake: classify the task in five minutes 67
17 Family A · Understand tasks 71
18 Family B · Fix tasks 74
19 Family C · Build tasks 78
20 Time: budgets, checkpoints, and when to ask 82

PART 6 — PROOF BY EXAMPLE

21 Worked example 1 · Fix 87
22 Worked example 2 · Understand 92
23 Worked example 3 · Build 96

PART 7 — HABITS

24 Steps 6 and 7 · Prove it and log it 102
25 Working with other people, and with AI 106

APPENDICES — REFERENCE

A VS Code over SSH: the step-by-step 110
B Command map: gdb and lldb 115
C Smoke test for your Linux machine 118
D Evidence and sources 121
E Glossary 125

Read this first

You get handed work in a code base you did not write. You have to fix something, add something, or just understand something. This book is a procedure for doing that quickly, accurately, and the same way every time.

The core idea in four lines

1. A debugger can only answer questions you can state as **data** and **conditions**.
2. Stating them is the hard part. That *is* understanding the problem.
3. So: first build confidence in the behavior, then translate it into a short written **Behavior Card** (what exists, what must be true, what changes it), then aim the debugger with it.
4. Every task follows the same **seven steps**. Only the first probe and the time split change with the kind of task.

The method on one page

#	Step	The question it answers	You produce	Done when
1	**Run it**	What does it actually do?	Command, input, and the output you saw	You can repeat the run on demand
2	**Understand it**	What is it *supposed* to do, and where do I work?	Purpose, premise, desired state, boundary	You pass the **Confidence Gate** (ch. 5)
3	**State it**	What are the data and conditions?	A filled **Behavior Card**	Every line is testable, unknowns are marked `?`
4	**Probe it**	What does the running program say?	Probes: conditional breakpoints, watchpoints, logs	Each probe has a prediction written first
5	**Pin it**	Which instance, which line?	One sentence: “X is Y because line L does not Z when C”	You can name the exact place and instance
6	**Prove it**	Did the change do what I claimed?	Minimal change + the same probes re-run	Probes that fired before are silent now
7	**Log it**	What will I (or the next person) need?	Journal entry, saved probe scripts	Someone else could repeat it

RULE **Always start with step 1.** Run the program and look at what it does before you read anything else.

How to use this book

- **30 seconds.** Open the Field Cards (the separate sheets). Card 1 is the method. Card 2 is the blank Behavior Card.
- **90 minutes.** Read Parts 1 to 4, then one worked example in Part 6. That is enough to start using it.
- **On the job.** Use chapter 16 to classify the task, the matching playbook in chapters 17 to 19, and chapter 20 to split your time.
- **Practice.** The `lab/` folder has a small program with a real bug. Three worked examples run on it.

Marks you will see

Mark	Meaning
RAN	I ran this command and captured the output. (lldb, on macOS.)
MANUAL	Checked against the official GDB manual, **not executed** by me. Run the smoke test on your Linux box (Appendix C) before relying on it.
SEC	A claim about a source I could only read through a reliable secondary page. Details in Appendix D.

Commands are written for **gdb first**. The lldb equivalent is given where I tested it.

What this book does not do

- It does not teach assembly or memory layout. You said you do not need that yet; chapter 11 has one page of “just enough” for pointers.
- It does not replace profilers or sanitizers. It says when to reach for them.
- It does not promise that hard problems become easy. It makes your time visible and your guessing small.
- Where the evidence is thin or a quote could not be verified, it says so. Appendix D lists every source and its status.

PART 1

The idea

Why a debugger is only as good as your understanding, what the best engineers agree on, and the seven steps this book is built from.

CHAPTER 1

The tool is only as good as the question

A debugger does not find bugs. It answers questions. If you cannot state the question as data and conditions, you cannot ask it.

The moment this book starts from

You described it yourself. You are placed on a real code base. The task touches a class with a hundred instances of a variable. You are sure the one you want is in there, but it appears in several methods, connects to pointers, and passes in and out of a messaging system. You trace imports and re-read the same functions, and nothing converges.

Then you learned breakpoints and watchpoints, and for a moment it felt solved. Then the second realization: **the tool works only as well as you understand the problem.** You do not know which instance. You do not know which condition. So you cannot tell the tool where to look.

That second realization is correct, and it is the thesis of this book.

What a debugger is, in one sentence

A debugger is a machine for answering questions shaped like this:

> *At this **place**, when this **condition** is true, what is the value of this **data**?*

Every command you have learned is one of those three things, or a way of combining them.

You want to say	The debugger's word	Which part is it?
"Stop in this function"	breakpoint	**place**
"…but only when `count > 100`"	condition (`break … if …`)	**condition**
"Tell me when this value changes"	watchpoint	**data**
"Show me the object"	`print *this`	**data**

The tool supplies the machinery. **You supply the place, the condition, and the data.** And you only know those if you understand what the code is supposed to do.

What the evidence says

EVIDENCE **Experts say the condition *is* the understanding.** In a study of how developers debug (survey of 176 developers, plus instrumentation of 458 developers' IDE use), two expert interviewees said that specifying a condition is "fuzzy" and that "once one knows the condition, one almost automatically understands the problem. Then, there is no need for the conditional breakpoint anymore." (Beller et al., ICSE 2018.) That is your insight, from people who debug for a living.

EVIDENCE **Most people never use the powerful features.** The same study observed that developers "did not run the debugger in 91% of IDE sessions", and that "while many developers knew and used conditional breakpoints, they were widely ignorant of hit counts and the debugger's other more advanced functions." It was Java in Eclipse and IntelliJ, so treat the direction as meaningful, not the exact percentages.

EVIDENCE **A correct early hypothesis predicts success.** In a controlled experiment with 20 developers, "having a correct hypothesis early strongly predicted later success", developers held about two hypotheses per bug, and being *shown the fault location* did not help them debug better. Being offered a plausible *hypothesis* did (Alaboudi and LaToza, 2020; small sample). Knowing *where* is not enough. You need to know *what you expect to be true there*.

EVIDENCE **Understanding is where the time goes.** A field study of 78 professional developers over 3,148 hours found they spend about 58% of their time on program comprehension (Xia et al., IEEE TSE 2018). The average hides large differences between tasks, but it says that the part of the job you are least trained for is the biggest part.

Three ways people get stuck without the translation

1. **Guess and print.** Add `printf`, rebuild, run, read, repeat. It can work, but every rebuild is slow, it changes the program, and each print answers a question you did not choose carefully.
2. **Read in circles.** Re-read the same functions hoping that understanding appears. Nothing in the code tells you which part matters.

3. **Step through the loop.** Fine for a program that runs and ends. Useless in an endless event loop, where stepping changes the timing and the state you are trying to observe, and where the thing you want happens on loop iteration 40,000.

The way out is the same for all three: **write down what should be true, in terms of data and conditions, before you touch the tool.**

What you get from this book

- A **process**: seven steps that work for any task (ch. 3).
- A **language**: declare, define, implement, turned into a one-page Behavior Card (ch. 6 and 7).
- **Probes**: conditional breakpoints, watchpoints, logging and counting, matched to the question you are asking (Part 4).
- A **classification** of tasks so you start the right way and budget your time (Part 5).
- A **lab**, so you can practice on a real bug before you need it at work.

TRY IT Take the last task you struggled with. In two sentences, write what the code *should* have done and what it *did*. If you cannot name a variable or a condition in those sentences, you have found exactly where the time went.

Check yourself

- Can you state a debugger's job as place, condition, data?
- For your current task, can you name one variable, one place, and one condition? If not, which of the three is missing?

CHAPTER 2

What the best engineers agree on

You asked how to use workflows developed by the best engineers. They disagree on tools and agree on process. Here are seven convictions that show up again and again, and what each became in this book.

Seven convictions

1. Understand before you act

George Pólya's *How to Solve It* (1945) puts "understand the problem" first. The checklist asks for **the unknown, the data, and the condition** that links them. Peter Naur argued in 1985 that a program is really the *theory* held by the people who built it: someone who has the theory can "explain how the solution relates to the affairs of the world," "support the actual program text with a justification," and "respond constructively to any demand for a modification." Modifications made "without a proper grasp of the underlying theory" are what make code decay. David Agans's first debugging rule is "Understand the System."

In this book: step 2 and the *Confidence Gate* (ch. 5). Pólya's unknown, data, condition become the Behavior Card (ch. 6).

2. Look. Don't guess.

Agans's third rule is "Quit Thinking and Look." He is not against guessing: "guess only to focus the search. You still have to confirm that your guess is correct by seeing the failure." Andreas Zeller's *Why Programs Fail* turns this into the scientific method: observe, form a hypothesis, make a *prediction*, test it with an experiment, refine or replace. Rob Pike's first rule of programming is "You can't tell where a program is going to spend its time." Google's SRE book describes troubleshooting as the hypothetico-deductive method: compare "the observed state of the system against our theories to find confirming or disconfirming evidence."

In this book: step 4. **Every probe has a prediction written before you run it.**

3. Data first

Pike's fifth rule: "If you've chosen the right data structures and organized things well, the algorithms will almost always be self-evident." He credits Fred Brooks. Brooks's version (chapter 9 of *The Mythical Man-Month*) says that if you show the tables, the flowcharts are rarely needed.

Linus Torvalds said the difference between good and bad programmers is that good ones "worry about data structures and their relationships" (git mailing list, 2006, as reprinted by LWN).

In this book: the Behavior Card starts with **data**, not functions. You find the code by following the data (step 5).

4. A behavior is a sequence of states. A bug is a state that breaks a rule.

Leslie Lamport's approach to specification (TLA+) describes a system as an initial state and a rule for what steps are allowed, and states properties as *invariants* (things that must always hold). Hoare and Meyer give the same idea in program form: `{P} C {Q}`, "if P holds before code C runs, Q holds after", and class invariants that must hold whenever an object can be observed. Zeller's model of a failure is a chain: a **defect** in code causes an **infection** (a bad program state), which propagates to an observable **failure**.

In this book: *Declare, Define, Implement* (ch. 6) is exactly this. You are looking for the **first bad state**, not the visible symptom.

5. Work in experiments

Zeller's debugging loop and the SRE book agree: hypotheses are cheap and wrong ones are expected. What matters is that each can be *falsified quickly*. Alaboudi and LaToza found that incorrect hypotheses "can cause developers to investigate irrelevant information and block progress", so a wrong hypothesis you keep without testing is expensive.

In this book: the Hypothesis Ledger (ch. 24) and the *counterfactual* probe, where you change a value by hand in the debugger to test a hypothesis *before* you edit any code.

6. Change one thing at a time, and keep a trail

Agans: "write down what you did, what order you did it in, and what happened as a result." And do not change the mechanism while you are measuring it.

In this book: step 7, the journal, and the preference for **probes over edits** while you are still investigating.

7. Shrink the problem, and follow your slice

Divide and conquer (Agans), bisection and "what touched it last" (SRE), and, from the talk that started this project, *follow the data instead of the control flow*: "most of us don't get the luxury of four years to understand the codebase before we're given our first bug to solve" (Mike Shah and Chris Croft-White, C++ on Sea 2025). Studies of how people read unfamiliar code describe

two strategies: reading systematically, and reading only what the task needs. Systematic readers did better on small programs; experts on large programs read “as needed”, driven by their goal (as described in a 2023 survey of the research).

In this book: the *slice rule* (ch. 5). Read everything that touches your data, and treat the rest as a black box with a stated assumption.

Where this book adds something

The authorities above give you principles. They do not tell you **how to get from “the link drops when it should not” to** `break file.cpp:81 if e.type == Heartbeat && missed_ != 0`. This book adds:

Gap	What fills it
Turning a human description of behavior into debugger input	The Behavior Card and the pattern catalog (ch. 6 to 8)
Choosing among breakpoint, condition, watchpoint, log, count, rewind	The probe chooser (ch. 9)
Finding *which instance* and *which line*	Pin it (ch. 15)
Starting the right way for the kind of task, and budgeting time	Intake, three families, time budgets (Part 5)

“What about innovation?”

You asked for a process that leads to results *and even innovation*. Here is the honest version.

What can be supported. Faster loops between a question and evidence mean you ask more questions. Bret Victor’s 2012 talk “Inventing on Principle” argues that creators need an immediate connection to what they create (known to me through secondary write-ups only). In the research on question-driven debugging, Ko and Myers’s *Whyline* prototype “reduced debugging time by nearly a factor of 8” in user tests (six scenarios in a teaching environment, so an upper bound and not a forecast).

What cannot be claimed. No study shows that this method produces innovation. The argument is narrower and more useful: the main thing that stops people experimenting in unfamiliar code is that every experiment is slow and scary. A process that makes experiments fast and cheap removes that barrier. What you build with the freedom is yours.

NOTE **On printf.** Beller et al. found `printf` debugging remains popular, and it is not wrong. In this book, a print is one kind of probe (LOG). The skill is picking the cheapest probe that answers your question. For long-running loops, debugger probes win because you can add, change, and remove them without rebuilding and without losing the running state.

Check yourself

- Which of the seven convictions do you currently skip under time pressure?
- What is the difference between a *failure* and an *infection*, and which one do you look for first?

CHAPTER 3

The seven steps

One flow for every task. Bug, feature, or “I just need to understand this” all go through the same seven steps. What changes is the first probe and how you split your time.

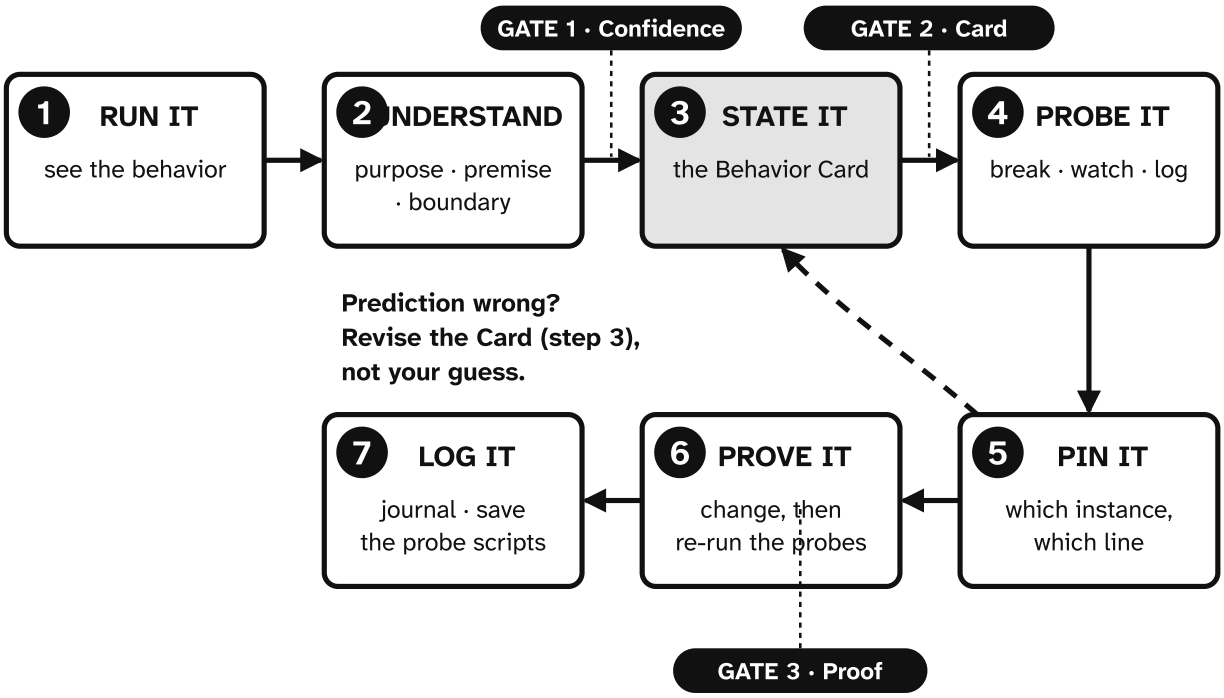


Figure 3.1. The seven steps and three gates. The dashed arrow is the loop you will use most: a probe disagrees with your prediction, so you revise the Card.

The steps in plain language

1. Run it. Build with debug symbols, run the program, and watch what it does. Write down the command, the input, and the output you saw. This is always first. You are looking at behavior, not code.

2. Understand it. Decide what the code is *for* and what “working” looks like, then decide which part of the system is yours. This is at least half reading the problem conceptually. If you do not understand the purpose, the premise, and the desired state, you cannot turn them into data and conditions.

3. State it. Translate the behavior into the three building blocks: **Declare** (what exists), **Define** (what must be true), **Implement** (what changes it). Write them on the **Behavior Card**. Anything you do not know yet becomes a `?` on the card.

4. Probe it. Aim the debugger with the card. Each `?` and each condition becomes a probe: a conditional breakpoint, a watchpoint, a counter, a log line. Write your prediction first.

5. Pin it. Use the probes to find the *first bad state* (for a bug) or the *change points* (for a feature): the exact instance and line. End with one sentence naming it.

6. Prove it. Test your sentence cheaply (change a value by hand in the debugger), make the smallest real change, then run the *same probes* again. They should now be silent. You must also have seen them fire on the old code.

7. Log it. Record what you did, what you learned, and save the probe scripts. This is how the next task gets faster.

The three gates

A gate is a pass/fail check. If you cannot pass, do not move on. Going on anyway is how the time disappears.

> **GATE** **Gate 1: Confidence (after step 2).** Can you *explain* the behavior in three plain sentences? Can you *predict* the next thing the program will print? Can you *locate* the module, the data, and a candidate function? (Chapter 5.)

> **GATE** **Gate 2: Card (after step 3).** Is every line testable? Does it name data, a place, and a condition? Are unknowns marked `?` with a plan for each? (Chapter 6.)

> **GATE** **Gate 3: Proof (after step 6).** Did you see the probe fire on the broken code and stay silent on the fixed code? Does the original run now behave? (Chapter 24.)

Why this order

- Step 1 before step 2 because you can only understand a system you have watched. Reading code first is how you get lost in a code base.

- Step 2 before step 3 because the card needs *meaning* (purpose, expected values). You cannot invent expected values.
- Step 3 before step 4 because the debugger needs *names and conditions*. Without them you stop at random places.
- Step 5 before step 6 because a fix applied before you can name the exact place is a guess that happens to compile.

The same steps, three kinds of task

Step	Understand (Family A)	Fix (Family B)	Build (Family C)
1 Run	See the behavior you must explain	Reproduce the bug	See the nearest existing behavior
2 Understand	Purpose and boundary	What should have happened	Desired new behavior and where it lives
3 State	State machine or data path	Expected vs observed state	New behavior as before/after states
4 Probe	Log who touches the data	Conditional stop at the bad state	Log writers and readers near the change
5 Pin	The map is complete	First bad state, exact line	Every place the change must go
6 Prove	Explain it back; predictions hold	Probe silent after fix	New behavior observed; old behavior unchanged
7 Log	Diagram and summary	Cause and fix	Change points and tests

Time

Where does the time go? Measured averages say more than half of a developer's time is comprehension. Do not budget the steps equally. The starting defaults (chapter 20) put roughly **45 to 60 percent of the time into steps 1 to 3**, because that is what makes steps 4 to 6 fast. These percentages are a starting proposal from this book, **not a research result**. Calibrate them with your own journal.

Check yourself

- Which step do you usually skip? What did it cost last time?
- State Gate 1 in your own words, without looking.

PART 2

Understand

Steps 1 and 2. Watch the program, then build enough confidence in what it should do that you can name the data and conditions.

CHAPTER 4

Step 1 · Run it

Before you read code, before you open a debugger, run the program and look at what it does. This is the cheapest probe there is.

Why it comes first

Mike Shah and Chris Croft-White open their method for unfamiliar code bases with “Step 1: run your program.” David Agans’s second rule is “Make It Fail”: you cannot debug what you cannot produce. Andreas Zeller’s TRAFFIC steps start with *Track* and *Reproduce*. Three sources, one instruction. A high-level feel for what the program actually does is the foundation everything else stands on.

For a **bug**, running it means reproducing the failure. For a **feature**, it means seeing the nearest existing behavior. For an **understanding** task, it means seeing the behavior you must explain.

Build for the debugger

Debuggers show you names only if the program carries them.

Build setting	What it does	If you skip it
`-g`	Adds symbols: variable names, types, line numbers	You see addresses, not names
`-O0`	No optimization	Variables show `<optimized out>`, lines jump around
CMake: `-DCMAKE_BUILD_TYPE=Debug`	Sets both for you	Release builds usually have neither

Check that symbols are present before you start:

```
gdb -batch -ex "list main" ./yourprogram      # prints source lines if symbols are found
```

If your project builds in release mode only, ask how to get a debug build. Never debug a different binary from the one that shows the problem without saying so in your notes.

What to write down: the Run Record

Three minutes, and it saves hours.

```
RUN RECORD
Command:   ./gateway --max 400
Directory: lab/
Input/config: none (deterministic event source)
Build:     -O0 -g, commit abc123
Output saved to: run1.log
What I expected: ...
What I saw:      ...  (copy the exact line)
First thing that looks wrong: ...
```

Make it repeatable

1. **Fixed input.** Same data, same seed, same config every run.
2. **Finite run.** An endless loop needs a way to stop: a `--max N` flag, `timeout 20 ./prog`, or Ctrl-C after N seconds. Save the output.
3. **Run it twice and compare.**

```
./gateway --max 400 > run1.log 2>&1
./gateway --max 400 > run2.log 2>&1
diff run1.log run2.log && echo "deterministic"
```

If the two runs differ, your first discovery is that the behavior is **non-deterministic** (timing, threads, uninitialized data). That changes the plan: see type B7 in chapter 18.

Read the output as a stream of state

Do not just skim. Look for four things:

- **Transitions.** Lines that say something changed (`Up -> Degraded`).
- **Counts.** Numbers that should add up or should not grow.
- **Timing.** What happened *after* what, and how long it took.
- **The first oddity.** Mark the earliest line that looks wrong. Everything before it is probably fine.

TRY IT **Lab.** Build and run the practice program. Read the output. Before you open any code, write one sentence about what looks wrong and why.

```
cd lab && make && ./gateway --max 400
```

This is what comes out. **RAN**

```
[t=  3] Down     -> Up       (missed=0 good=3)
[t= 23] Up       -> Degraded (missed=4 good=0)
[t= 27] Degraded -> Down     (missed=8 good=0)
[t= 29] Down     -> Up       (missed=0 good=3)
[t=123] Up       -> Degraded (missed=4 good=0)
[t=127] Degraded -> Down     (missed=8 good=0)
[t=134] Down     -> Up       (missed=0 good=3)
final: state=Up forwarded=54 dropped=6 data_seen=153
```

Seven state changes. You do not know yet which ones are wrong. You cannot know without the requirements, and that is step 2. But you can already say something useful: “at t=27 the link went Down only four ticks after it went Degraded.” Keep that line; chapter 21 comes back to it.

Anchor on `main`

When the code base is large, find where it starts. From `main` you often learn the architecture: is there an event loop, are plugins loaded, what happens before and after the loop.

```
rg -n "int main" src/            # find the entry point
rg -n "while *\(" src/ | head    # find loops near it
```

NOTE `cloc` counts lines of code per language. Run it once to size the job. A million lines is roughly 18,000 pages. You will not read it all, and chapter 5 explains why you do not need to.

When the program never ends

An event-driven system is not supposed to finish. For step 1 you only need a **window** of behavior:

- Add or use an event limit (`--max`).
- Or capture a fixed time: `timeout 30 ./prog > run1.log 2>&1`.
- Or Ctrl-C when the symptom appears, and note the last lines.

Part 4 shows how to look *inside* such a loop without stepping through it.

Check yourself

- Can you run it, on demand, the same way twice?
- Did you save the exact line that looks wrong?
- Is the build a debug build (`-O0 -g`)?

CHAPTER 5

Step 2 · Understand it

At least half of any new task is reading the problem conceptually. If you do not know the purpose, the premise, and the desired state, you cannot turn them into data and conditions. This chapter makes that reading fast, bounded, and testable.

What “understand” means here

Not “I have read every function.” It means you hold a small **theory** of the part you work on, in Peter Naur’s sense: you can explain it, justify it, and respond to a change request. It is tested by **Gate 1** at the end of this chapter.

Five questions

Answer each in one or two lines. Short answers are better than long ones.

#	Question	Prompt	Practice program (the gateway)
1	**Purpose**	Why does this exist? Who or what uses its output?	Decide if the link to a peer is usable, so data is forwarded only when it is.
2	**Premise**	What must be true for it to work? What are the inputs and assumptions?	Events arrive in order: Tick, then Heartbeat, then Data. Heartbeats arrive every tick when healthy.
3	**Desired state**	What does “working” look like in things I can observe?	Rules R1 to R6 in `gateway.cpp`. For example R5: a link that is receiving heartbeats must not be declared Down.
4	**Boundary**	Which part is mine? What can I treat as a black box?	`LinkMonitor` and the data it holds. The event source and the Gateway are black boxes.

#	Question	Prompt	Practice program (the gateway)
5	**Data path**	Where does the data enter, change, and leave?	Event in, `missed_` and `good_` change, `state_` out, read by `Gateway::dispatch`.

Where to get the answers, fastest first

Source	What it gives you	Typical cost
The run you just did (step 1)	Real behavior	Done
Ticket, requirement, acceptance criteria	Desired state, in words	5 min
Declarations: headers, class definitions, data members, enums	The nouns: what data exists and its types	10 min
Existing tests	Executable examples of expected behavior	10 min
Diagrams and design docs	Boundary and data path	10 to 20 min
History: `git log -S missed_ --path` and `git blame`	"What touched it last" (Google SRE's phrase)	5 min
A person	What is *not* written down	One good question

Read **declarations before implementations**. The header tells you what exists. The bodies tell you what happens. That is the Declare and Implement split you will use in chapter 6, applied to reading.

> **RULE** **Ask with a hypothesis, not just a symptom.** "I think `missed_` should reset on every heartbeat but the Degraded path doesn't. Is that intended?" gets better answers than "the link is flaky." (Alaboudi and LaToza found that being offered hypotheses helped people succeed far more than being shown locations.)

How much is too much? The slice rule

You asked this directly: how much information is too much, how much too little, and if my module has no explicit messaging with others, can I ignore the rest?

> **RULE** **The slice rule.** Read *completely* everything that reads or writes the data on your card (one hop). Treat everything else as a **black box with a written assumption** (“`EventSource` delivers Tick before Heartbeat within a tick”). Stop reading when you can fill the Behavior Card.

This is our synthesis of two research findings: systematic readers did better on small programs, while experts on large ones read “as needed”, driven by their goal (as described in a 2023 survey of the literature). So: be systematic *inside* a small slice, and as-needed outside it.

Test a boundary. Ask: *if I replaced module X with a stub that returns Y, would my task change?* If no, X is a black box. If yes, X is inside your slice.

Signs you have read too little:

- You cannot predict the next lines of output.
- The same question keeps coming back after you “read it”.
- You are changing a value and the result surprises you.

Signs you have read too much:

- You are reading code that never touches the data on your card.
- You are drawing a diagram of the whole system for a task that touches one class.
- Half of your Understand time is gone and the card is still empty. Stop reading. Go probe (Agans: “quit thinking and look”).

“My module doesn’t talk to the others. Can I ignore them?”

Not by messaging alone. Modules also interact through **shared state**. Check these, in this order:

1. Globals, statics, singletons
2. Shared queues, buffers, caches
3. Files, databases, configuration
4. Hardware registers and memory-mapped I/O (embedded)
5. Timing: priorities, timers, who runs when
6. The same lock or the same resource

If none of these intersects the data on your card, you can treat your task as pure logic. You can also **test it with the debugger**: watch the card’s data during a full run and log who writes it (chapter 11). If every writer you see is inside your slice, the slice is closed. If an unexpected function appears in a backtrace, widen the slice by exactly that one function.

If there are no diagrams, draw three

Ten minutes, on paper. These are the minimum.

```
1. CONTEXT: who talks to whom          2. DATA PATH: one value's journey
   +-------+     +-------+                  input -> [stage A] -> [stage B] -> output
   | mine  |<--->| other |                            ^ where can it change?
   +-------+     +-------+                            ^ who else touches it?

3. STATE x EVENT TABLE (best form for event-driven code)
   | state \ event | Tick | Heartbeat | Data |
```

For event-driven systems the **state by event table** is the most valuable of the three, because it lists every cell you must reason about. Build it from the **requirements** (the desired state), not from the code. Then the code has something to be compared against. Here it is for the practice program, from rules R1 to R6:

State Event	Tick	Heartbeat	Data
Down	`missed++`	`missed=0`; `good++`; if `good>=3` go Up (R1)	count; drop (R6)
Up	`missed++`; if `missed>=4` go Degraded (R2)	`missed=0` (R5)	count; forward (R6)
Degraded	`missed++`; if `missed>=8` go Down (R3)	`missed=0` (R5); `good++`; if `good>=2` go Up (R4)	count; drop (R6)

Every cell is a claim you can check. In chapter 21 the probes will show that the code disagrees with exactly one of them.

Tools that help when you want them: your IDE's *Find All References* and *Call Hierarchy*, `rg` for visible strings, and Doxygen with Graphviz for call graphs (`CALL_GRAPH = YES`). A live `bt` in the debugger is also a call graph for one moment in time. MANUAL

Gate 1: the Confidence Gate

> **GATE** **Pass all three before you leave step 2.**
>
> **Explain.** Say the behavior in three plain sentences, with no “somehow” or “I think it kind of”. Include why it exists.
>
> **Predict.** Write down the next two lines of output before running again. Run. If you were wrong, you do not understand it yet, and the *difference* is your next question.
>
> **Locate.** Name the module, one piece of data, and one candidate function where the behavior lives.

These are Naur’s three abilities (explain, justify, respond to a modification) as a pass/fail check.

Time-box it

Understand takes about a quarter to a third of a task’s time in the starting budgets of chapter 20. Spend **half of that reading**, then switch to probing. If the gate still fails after the full allocation, you are blocked on *knowledge*, not tools. Escalate (chapter 20).

Check yourself

- Which of the five questions can you not answer for your current task?
- What is the boundary of your slice, and what assumption do you write down for each black box?

PART 3

State it

Step 3. Turn a human description of behavior into data and conditions, in the words a debugger understands.

CHAPTER 6

Step 3 · State it

You know the behavior. You do not yet know how to describe it as a state in a specific module, class, and method. This chapter gives you three building blocks and one page to write them on.

The translation problem

In your own words: “A bug is *we can’t do this*, or *the system is doing the opposite of what we want*. It is behavior. So how do we describe the behavior in code?” And for a feature: describe the new behavior in terms of what has already been declared, defined, and implemented. You cannot use a watchpoint or a conditional breakpoint until you can do that translation, because you are not yet speaking the tool’s language.

The translation has three parts.

Declare, Define, Implement

> **NOTE** These words come from C and C++, where they have exact meanings: a *declaration* introduces a name and a type, a *definition* creates the thing or gives it its value or body, and the *implementation* is the code that does the work. We borrow them with a slightly wider working meaning that maps onto what you write on the card.

Block	Working meaning	The question	Where you find it	Becomes, in the debugger
DECLARE	**What exists, and where.** The nouns: data, with names and types, and the places (module, class, function, file:line) they live.	*What are the things involved?*	Headers, class definitions, data members, enums. `ptype`, `info scope`.	What to `print` and what to `watch`
DEFINE	**What must be true, and when.** Initial values, legal ranges, the condition that turns the behavior on or off, the expected value and the observed value.	*When is the behavior right or wrong?*	Requirements, tests, `assert`s, constants, comments, the run output.	The `if` in a conditional breakpoint or watchpoint
IMPLEMENT	**What changes it, and how.** The verbs: the code that writes the data (writers), reads it (readers), the branch that decides, the calls that carry it.	*What changes the data, and under what branch?*	Function bodies. A watchpoint log. `bt`.	Where to put a probe, and where the fix or feature goes

This is the same structure as the classic `{P} C {Q}` of Hoare and Meyer: if condition P holds before code C runs, condition Q holds after. **Declare** names the things inside P and Q. **Define** *is* P, Q, and the invariants. **Implement** is C. A bug is a state where Q (or an invariant) is false. The *infection* in Zeller's chain is the first such state.

The Behavior Card

One card per behavior. It fits on one page. A blank, full-size copy is Card 2 in the Field Cards.

BEHAVIOR CARD

Task: _____ **Date:** _____ **Family / type:** A □ B □ C □ ___

RUN (command and the exact line you saw)

EXPECTED (and who says so: ticket, spec, person)

PURPOSE in one sentence

DECLARE: what exists (name · type · file:line · which instance)

DEFINE: before/trigger (P)

after/expected (Q)

always true (I)

observed instead (Q′)

IMPLEMENT: writers

readers

deciding branch

unknowns **?**

PROBES: verb · command · *prediction* · if wrong

HYPOTHESES: H1

H2

The card, filled in for the practice program

Here is the card you would write after steps 1 and 2 for the false `Down` in the gateway. **RAN** means the run behind it is real. The probes it lists are run in chapter 21.

```
BEHAVIOR CARD — link falsely goes Down                Family B (Fix) · type B1
RUN       ./gateway --max 400
          "[t= 27] Degraded -> Down     (missed=8 good=0)"
EXPECTED  R5: a link that is receiving heartbeats must not be declared Down.
          (source: the requirement comment at the top of gateway.cpp)
PURPOSE   Decide link health from ticks and heartbeats, so data is forwarded
          only when the link is usable.

DECLARE   what exists
  D1  LinkMonitor::state_   enum LinkState {Down,Up,Degraded}   gateway.cpp:132   (one
instance)
  D2  LinkMonitor::missed_  int  ticks since the last heartbeat   gateway.cpp:133
  D3  LinkMonitor::good_    int  consecutive heartbeats           gateway.cpp:134
  D4  Event e               {type, tick, payload}  argument of LinkMonitor::on_event

DEFINE    what must be true
  P  (trigger)   a Heartbeat event is handled          e.type == Heartbeat
  Q  (expected)  missed_ == 0 afterwards                 (R5 and the Heartbeat column)
  I  (always)    after any Heartbeat is handled, missed_ == 0
  Q' (observed)  ?  at t=26 a Heartbeat arrived but the Down at t=27 says missed=8

IMPLEMENT what changes it
  writers of missed_   on_tick: ++ ; on_heartbeat: = 0 in SOME branches  (? which)
  readers of missed_   on_tick compares to 4 and 8 -> decides Degraded / Down
  ? Does every branch of on_heartbeat clear missed_?   -> probe P1

PROBES
  P1 FILTER      stop after on_event if e.type==Heartbeat && missed_!=0
                 predict: never fires.  If it fires: the invariant is broken, note
state.
  P2 EXPERIMENT  at that stop, set missed_ = 0 and continue
                 predict: Degraded -> Up at t=27 instead of Down.

HYPOTHESES
  H1  a Heartbeat while Degraded does not clear missed_.        status: open
```

Notice what the card does. It turned “the link drops when it should not” into a **named variable** (`missed_`), a **condition** (`e.type == Heartbeat && missed_ != 0`), and a **place** (the end of `on_event`). That is the input a debugger needs.

What makes a good card

- **Testable.** Every DEFINE line can be written as an expression that is true or false.
- **Named and located.** Each piece of data has a name, a type, and a file and line.

- **Expected value has a source.** You did not invent it. A ticket, a spec, a test, or a person said so.
- **Observed value is copied.** The exact line from the run, not a paraphrase.
- **Unknowns are marked** `?` and each `?` has a probe next to it (chapter 8).
- **One behavior per card.** Two symptoms get two cards.

The card for each kind of task

Only the DEFINE section changes in spirit.

Family	EXPECTED means	Q′ (observed) means
A. Understand	The behavior you must be able to explain	What you saw it do. The card is complete when Q is *explained*, not changed.
B. Fix	What the spec says should happen	The wrong state you saw
C. Build	The new behavior, written as before and after states	What happens today in the same situation (the nearest existing behavior)

GATE **Gate 2: the Card.** Before probing: does every DEFINE line have a testable expression? Does every piece of data have a location? Is every unknown a `?` with a plan? If not, you will be setting breakpoints at random.

Check yourself

- Write one DEFINE line for your current task as an expression a debugger could evaluate.
- Which part of your card is the weakest: the nouns, the conditions, or the verbs?

CHAPTER 7

From English to conditions

Most behavior descriptions fall into ten patterns. Each pattern has a logical form and a standard probe. Learn the patterns once and translation becomes lookup.

The pattern catalog

Find the phrase in the left column. Write the logic in the middle column on your card. Use the probe in the right column. `END` means “the last line of the function that changes the data”.

If the English says…	The logic	The probe
“This must **always** / **never** be true” (invariant)	`I` holds at every observation point	Conditional breakpoint at `END` of every function that changes the data: `break END if !(I)`
“**After** X, Y should hold” (postcondition)	`{P} X {Q}`	Breakpoint at X’s exit with `if !(Q)`. Or `finish` and read the result
“**When** X, do Y” (guard and action)	`if (G) A`	Breakpoint **at A** (did it run?), and at the line that tests G, showing G’s operands
“Y **didn’t happen**” / “why not?”	G was false, or never evaluated	Breakpoint at the decision, log each operand; count hits of the function (is it even reached?)
“Y happened but **shouldn’t have**”	A ran while G was false	Breakpoint at A with `if !(G_expected)`; `bt` shows which path
“**Only if** X” (precondition)	`P` required before `C`	Breakpoint at C’s entry with `if !(P)`
“The **Nth time**” / “after N events”	a counter	Ignore count, or a counter variable; a conditional on the counter
“**In this order**”	sequence S1 then S2	Log both with a sequence number; or **arm**: breakpoint 1 enables breakpoint 2
“It’s **stuck** / never finishes”	progress variable `p` must change	Interrupt and `bt`; tripwire on an iteration counter; a watchpoint on `p` that never fires

If the English says...	The logic	The probe
"Only for **this one** user / device / instance"	scope to one thing	Add `this == $target`, a thread, or a string compare to any condition above

Each probe is covered in Part 4. Here you only need to *choose* it from the card.

Worked translations

"The link should not go Down while heartbeats are arriving." It says *should not* and *while*. That is an **invariant** over the state. Find the data: `state_`, `missed_`. A heartbeat is an event. Rewrite: "after a heartbeat is handled, `missed_` is zero." Probe: stop at the end of `on_event` if that is false.

```
break gateway.cpp:81 if e.type == EventType::Heartbeat && missed_ != 0
```

"Nothing is logged when the link recovers." [illustrative] It says *nothing happened*. That is a **didn't happen**. The action A is the log call. The guard G is whatever decides to log. Probe: a breakpoint on the logging function (zero hits?). If it is never reached, stop at the decision above it and print the operands.

"The first alarm fires, but the second one never does." [illustrative] *The Nth time*, plus *didn't happen*. Count how many times the alarm function is reached, then compare against the guard. A counter works well here (chapter 10).

Writing conditions that work

- **Say it aloud.** `break … if e.type == Heartbeat && missed_ != 0` reads as "stop after handling a heartbeat where the counter was not cleared." If it does not read as the behavior, rewrite it.
- **Use names the debugger knows.** In C++, use scoped enum names (`EventType::Heartbeat`), member names, and `this->member`. **RAN** in lldb; standard in gdb. **MANUAL**
- **Parentheses around negations:** `!(a && b)`, not `!a && b`.
- **Strings:** `$_streq(name, "x")` in gdb (needs a Python-enabled gdb). **MANUAL** Comparing `char*` with `==` compares addresses, not text.
- **Avoid side effects and function calls in a condition.** The GDB manual says "Break conditions can have side effects, and may even call functions in your program", and recommends command lists for side effects. In lldb I tried `strstr(...)` in a condition and

got `'strstr' has unknown return type; cast the call to its declared return type`. **RAN** Calls are fragile and slow.

- **Know the cost.** In gdb the program stops every time the location is reached, then the condition is evaluated and, if false, it resumes. Cheap in a small loop, noticeable in a tight one. For remote targets, `set breakpoint condition-evaluation target` lets the target evaluate it (default is `auto`). **MANUAL**

TRAP **The condition that never fires.** Common causes: comparing floats with `==`; a typo that parses as a different variable; a condition placed on a line that is not reached; or a condition about the *wrong instance*. Before you trust silence, make sure the probe *can* fire. Temporarily loosen it, or test it on a known-bad case.

Choosing the *place*

The place matters as much as the condition.

- **For an invariant, put the probe at the end of the function that changes the data.** The data was just updated, so a violation is fresh and the backtrace points at the culprit.
- **For a postcondition, put it at the exit.**
- **For a "didn't happen", put it at the decision** (where the guard is tested), not at the action that never ran.
- **Put it where the data is in scope.** A condition can only use names visible at that line.

TRY IT Take the DEFINE section of your current card. For each line, find its row in the table, name the probe, and write the place. If a line does not fit any row, it is probably two behaviors. Split it.

Check yourself

- Which pattern is "the screen never updates"? (A *didn't happen*.)
- Where would you put the probe for "`count` must never exceed 10"? (At the end of every function that changes `count`.)

CHAPTER 8

When you do not know the names yet

If you are new to a code base, you do not know which function, which variable, or which condition. So your first task is investigating and finding the missing information. This chapter gives every `?` on the card a probe.

The discovery loop

You cannot write a condition about a variable you cannot name. So:

1. Write the card as far as you can. Mark each missing name `?`.
2. For each `?`, pick the discovery probe from the table below.
3. Run the probe. Write the answer into the card.
4. Repeat until no `?` blocks you.

Each probe is cheap and answers one question. None of them requires reading the whole system.

Discovery probes

The `?` is...	Probe	Command
Where is the code for this behavior?	Search for visible text from the output: log strings, error messages	`rg -n "recovered" src/`
	Break on the **output function** and read the backtrace (walk from effect to cause)	`break printf`, `run`, `bt`
	List functions matching a name pattern	`info functions Link` (gdb) / `image lookup -r -n Link` (lldb)
Who calls this function?	Break on it and look at the stack	`break F`, `run`, `bt`
	Log every caller, then count them	commands list below
Which data does it use?	Look at arguments, locals, and the object	`info args`, `info locals`, `print *this`
	Show a type's fields	`ptype LinkMonitor` (gdb) / `type lookup LinkMonitor` (lldb)

The `?` is…	Probe	Command
What values does it take?	Log the value each time, then tally	`dprintf`, then tally the output with `uniq -c`
Who changes this variable?	Watchpoint with a log (chapter 11)	`watch -l expr`
When does it run?	Count hits; log with a tick or timestamp	breakpoint `info breakpoints` hit counts

Break on the output

When you know *what the program says* but not *where it says it*, break on the thing that does the saying. **RAN**

```
(lldb) breakpoint set -n printf
(lldb) run --max 400
(lldb) bt 5
* frame #0: libsystem_c.dylib`printf
  frame #1: gateway`LinkMonitor::announce(this=…, from=Down) at gateway.cpp:128:5
  frame #2: gateway`LinkMonitor::on_heartbeat(this=…) at gateway.cpp:115:9
  frame #3: gateway`LinkMonitor::on_event(this=…, e=…) at gateway.cpp:78:34
  frame #4: gateway`Gateway::dispatch(this=…, e=…) at gateway.cpp:159:14
```

One command took you from a log line to the function that prints it, the function that decided to print it, and the loop that called both. In gdb it is the same idea. **MANUAL**

```
break printf
run --max 400
bt 5
```

Log every caller, then count

“Is this the only place it is called?” Run, log the callers, tally them. **MANUAL**

```
break LinkMonitor::on_event
commands
  silent
  bt 2
  continue
end
run --max 400
```

Save the output and run `grep "Gateway::" out.txt | sort | uniq -c`. You get the callers seen at runtime and how often. **Limit:** this only shows what *ran*. A path that was not exercised stays invisible, so complement it with a static search (`rg "on_event("`, *Find All References*). Chapter 22 does this for real.

Reading a backtrace

`bt` lists **frames**, innermost first. Frame `0` is where you are stopped. Frame 1 called frame `0`, and so on. For each frame you can look at its own arguments and locals (chapter 12).

```
#0  LinkMonitor::on_event   <- you are here
#1  Gateway::dispatch        <- who called you
#2  Gateway::run             <- and who called them
#3  main
```

A backtrace answers *how did I get here*, which is the control-flow question. The next question, *where did this data come from*, is answered with watchpoints.

When the code base is large

- Start from an **effect** you can see (output, a file, a message) and walk back.
- Start from a **function you already know** and walk out.
- Use the **slice rule** (chapter 5): you only need the code that touches your data.

> **NOTE** **Messaging systems.** When data goes out of a class through a message and comes back, the data path is broken in the code but not at runtime. Break on the *send* function and on the *receive* handler, log a distinctive field (a sequence number, an id) at both, and you can follow one message across the gap. Chapter 17 (type A4) uses the same idea.

Check yourself

- For each `?` on your card, which probe from the table will answer it?
- Why does a backtrace from an output call find code faster than reading?

PART 4

Probe it

Step 4, and the tools behind steps 5 and 6. How to look inside a program that never stops, and which probe answers which question.

CHAPTER 9

Debugging a loop that never ends

The basic recipe, `break main` then `next`, `next`, `next`, works for a program that runs and finishes. In a continuously running, event-driven system it fails. You need a different model.

Why stepping fails here

In your words: stepping through the loop can be useless, and can cause the state you are looking for to differ from what you expected.

- **The world keeps moving.** While you are stopped, real time passes. Timers expire, peers time out, buffers fill, a watchdog resets the device. When you resume, the system is not in the state it would have been in.
- **Iteration 40,000.** The thing you want happens deep into the run. Stepping gets you there in hours.
- **You change what you measure.** Agans's rule: use instrumentation to look at what is going wrong, but "don't change the mechanism; that's what's causing the failure."

The model: a stream of events

Stop thinking of the program as a path you walk. Think of it as a **stream of events** flowing through code that changes state. You do not follow the stream. You **tap** it.

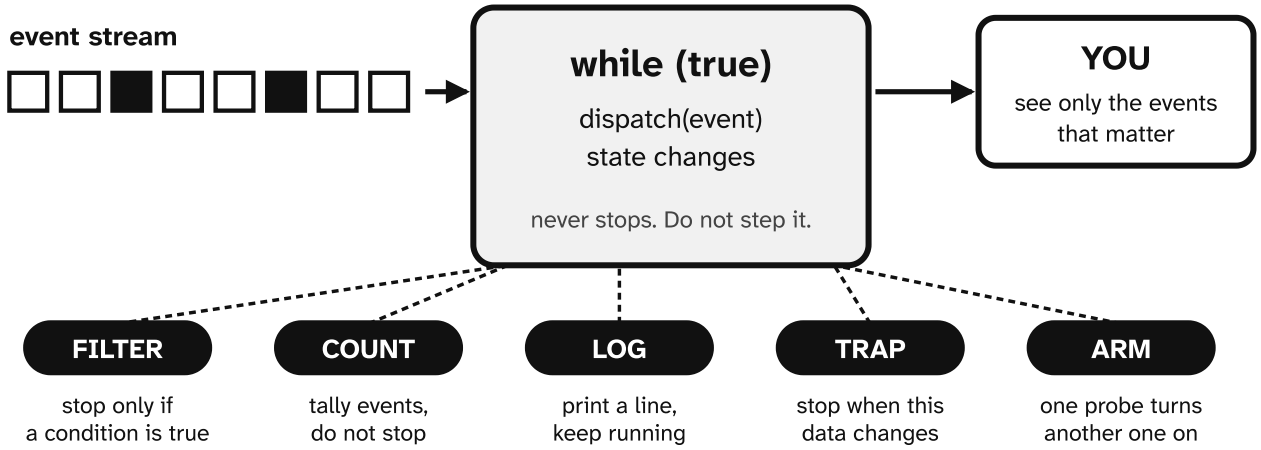


Figure 9.1. You do not step through an endless loop. You attach probes to it, and the probes decide which events reach you.

A probe is a small instruction to the debugger: *when this happens, do that*. You attach several and let the program run at full speed. The debugger interrupts only for events you chose.

Six verbs

Verb	What it does	The question it answers	gdb	lldb
FILTER	Stop only when a condition is true	*Show me the one event that breaks the rule.*	`break LOC if COND`	`breakpoint set … -c 'COND'` **RAN**
COUNT	Tally events without stopping	*How often? Is it the 3rd time?*	a convenience variable, in a `commands` list	Python callback, return False **RAN**
LOG	Print a line and keep running	*What values did it take over time?*	`dprintf`, or a `commands` list that ends in `continue`	Python callback, return False **RAN**
TRAP	Stop when a piece of data changes	*Who writes this?*	`watch`, `rwatch`, `awatch`	`watchpoint set variable` **RAN**
ARM	One probe switches another on	*Only after X has happened.*	`disable N`, then `enable N` in a `commands` list	Python callback calling `SetEnabled` (untested)

Verb	What it does	The question it answers	gdb	lldb
REWIND	Go back in time from a bad state	*The state is bad now. What made it so?*	`reverse-continue` with `rr` or `record`	no equivalent used here

Everything in the rest of Part 4 is a detail of one of these six verbs.

Fundamentals you may be missing

You asked whether there are other basic strategies besides breakpoints and watchpoints. Here is the whole list, in the order they tend to pay off. The first two you already have.

#	Strategy	Why it matters in a loop
1	Conditional breakpoint	You have it. The workhorse.
2	Watchpoint	You have it. Follows data, not code.
3	**Ignore count / hit count**	"Skip the first 1,000 passes", or "stop on the 25th."
4	**Logging without stopping** (`dprintf`, `commands` + `silent`)	Observe the stream with no stop and no rebuild. The debugger version of `printf`.
5	**One-shot breakpoints** (`tbreak`, `enable once`)	Stop once, then stay out of the way.
6	**Counters** (convenience variables)	Measure frequency and rank causes.
7	**Arming**	Watch a variable only after a trigger event. Cuts noise by orders of magnitude.
8	**Scope filters**: caller, instance, thread	Isolate "this one object" or "only via this path."
9	**Read and access watchpoints** (`rwatch`, `awatch`)	Find the code that *consumes* a value, not just the code that writes it.
10	**Catchpoints** (`catch throw`, `catch signal`, `catch syscall`)	Stop on an event class, not a line.
11	**Change a value by hand** (`set var`)	Test a hypothesis in seconds, before touching source.
12	**Reverse execution / record and replay**	Go from "bad now" back to "who wrote it."

#	Strategy	Why it matters in a loop
13	**Post-mortem** (core dumps)	Examine a crash you cannot reproduce at your desk.
14	**Scripts and the Python API**	Make probes repeatable and shareable (chapter 14).
15	**Non-debugger tools**: assertions, structured logs, hardware trace	Some systems cannot be halted at all. See the box below.

RULE **The three that change your week most are #4 (log without stopping), #7 (arming), and #11 (change a value by hand).** Learn those first.

TRAP **On real hardware, a stop halts the processor.** A breakpoint that stops and resumes still pauses the CPU for a moment, which may disturb timing, peripherals, or a watchdog. Use `dprintf` and logging probes sparingly on time-critical code. Non-intrusive tracing exists (hardware trace and tools built on it), but I have not verified specifics for them and they are outside this book. Ask your hardware team what your target supports.

One question per probe

Every probe answers a single question and carries a prediction. Write them on the card as you go.

```
PROBE CARD
Q:          Does missed_ ever stay non-zero after a heartbeat?
Verb:       FILTER
Place:      end of LinkMonitor::on_event (gateway.cpp:81)
Condition:  e.type == Heartbeat && missed_ != 0
Predict:    never fires
If it fires: the invariant is broken. Note state_, tick, and what comes next.
If silent:   the probe may be wrong. Check that it can fire (chapter 7).
```

If you cannot write the prediction, you are not ready to run the probe. Go back to the card.

The probe chooser

Start from what you know, not from the command you remember.

You know...	You want...	Use
A **place**, and the data looks wrong there	The first bad moment	FILTER
A place that runs thousands of times	The Nth time, or after a warm-up	Ignore count, or COUNT plus a conditional
A place, and you do not want to stop	Values over time	LOG
A **piece of data**, not a place	Who changes it	TRAP (`watch -l`)
A piece of data, and only bad values matter	Filter the writes	TRAP with a condition
A piece of data, and you want its **consumers**	Who reads it	`rwatch` or `awatch`
Something rare that happens **after** an event	Only look then	ARM
An event class (exception, signal, system call)	Stop when it occurs	`catch …`
The state is bad **now**, the cause was earlier	Go back	REWIND
A crash you cannot reproduce live	The state at death	Core dump
A guess about the cause	Test it without editing	`set var`, then `continue`

Check yourself

- Why can stepping through an event loop give you a misleading picture?
- Name the six verbs. Which one do you use when you know the data but not the place?

CHAPTER 10

Breakpoints and conditions

A breakpoint is a place. Everything else in this chapter is how to make that place stop only when it matters, or not stop at all.

Commands are gdb first. lldb versions are shown where I ran them. **RAN** marks lldb output I captured; **MANUAL** marks gdb syntax checked against the manual only.

The basics

Goal	gdb	lldb
Stop at a function	`break Gateway::run`	`breakpoint set -n Gateway::run` **RAN**
Stop at a file and line	`break gateway.cpp:81`	`breakpoint set -f gateway.cpp -l 81` **RAN**
List breakpoints and **hit counts**	`info breakpoints`	`breakpoint list` **RAN**
Turn off, on, remove	`disable 2`, `enable 2`, `delete 2`	`breakpoint disable 2`, `enable 2`, `delete 2`
Continue, finish the function	`continue`, `finish`	`continue`, `finish`

`finish` runs until the current function returns and prints the returned value. It is the quickest way to ask “what did this call give back?”

FILTER: conditions

```
break gateway.cpp:81 if e.type == EventType::Heartbeat && missed_ != 0       [MANUAL]
condition 1 missed_ > 5        # add or change a condition on breakpoint 1
condition 1                    # remove it
```

lldb **RAN**:

```
breakpoint set -f gateway.cpp -l 81 -c 'e.type == EventType::Heartbeat && missed_ !=
0'
```

This stopped exactly at t=26 with `state_ = Degraded, missed_ = 7`.

TRAP **An ignore count switches the condition off.** The manual says: “If a breakpoint has a positive ignore count and a condition, the condition is not checked.” So `ignore 1 100` plus a condition means “skip the first 100 times the location is *reached*”, not “skip the first 100 times the condition is true.” If you need the second, count with a variable (below).

Ignore counts: skip the warm-up

```
ignore 1 1000        # breakpoint 1: let it pass the next 1000 times      [MANUAL]
```

lldb **RAN**: `breakpoint set -n LinkMonitor::on_tick -i 3` skipped three hits and stopped on the fourth. `breakpoint list` then showed `hit count = 4`.

Hit counts are free information. After a run, `info breakpoints` tells you how many times every location was reached, including breakpoints that never stopped you.

One-shot breakpoints

Goal	gdb	lldb
Stop once, then remove	`tbreak LOC`	`breakpoint set … -o true` **RAN**
Stop once, keep it disabled	`enable once N`	
Stop the next C times	`enable count C N`	

Use one-shots to get *into* the right context (for example, to read an instance’s address) without leaving a breakpoint behind.

LOG: print without stopping

`dprintf` is a breakpoint that prints and continues. **MANUAL**

```
dprintf gateway.cpp:108,"heartbeat t=%d state=%d missed=%d\n", now_, (int)state_,
missed_
```

A command list gives you more control. The manual’s own example:

```
break foo if x>0
commands
silent
printf "x is %d\n",x
cont
end
```

Rules from the manual: `silent` must be the first command; “any other commands in the command list, after a command that resumes execution, are ignored”, so put `continue` **last**. Use `echo`, `printf`, or `output` for exact formatting.

lldb has no `silent` command list that prints reliably in batch mode, so I used **Python callbacks**. A callback returns `False` to keep running (a log line) or `True` to stop. **RAN**

```
def log_state_write(frame, bp_loc, internal_dict):
    this = frame.FindVariable("this")
    state = this.GetChildMemberWithName("state_").GetValue()
    print("  state_ written -> %s" % state)
    return False          # keep running
```

Attach with `breakpoint command add -F probes.log_state_write 1` (or `watchpoint command add -F …`). The full file is `lab/lldb/probes.py`.

COUNT: tally with a variable

```
set $n = 0
break LinkMonitor::on_heartbeat
commands
  silent
  set $n = $n + 1
  continue
end
break Gateway::summary
commands
  silent
  printf "heartbeats handled: %d\n", $n
end
```

MANUAL The same idea with an lldb callback printed `heartbeats handled: 160` for `--max 400`. **RAN**

A convenience variable (`$name`) is created when you first assign it and is void until then. `init-if-undefined $n = 0` sets it only if it does not exist yet.

Scope filters: only this caller, instance, or thread

By caller (needs a Python-enabled gdb): MANUAL

```
break LinkMonitor::on_event if $_caller_matches(".*flush_burst")
```

`$_caller_is(name)` checks the immediate caller. `$_caller_matches(regex)` matches a pattern. `$_any_caller_is(name, N)` and `$_any_caller_matches(regex, N)` check all frames up to N levels. The manual does not say whether the regex is anchored at the start, so write patterns like `".*flush_burst"` and confirm the first time you use it. If you have no Python, break at the **call site** instead (`break gateway.cpp:170`).

By instance. When there are many objects of one class, pick one by its address and condition on it:

```
break LinkMonitor::on_heartbeat
run
# stopped in the instance you care about:
set $target = this
condition 1 this == $target                                          [MANUAL]
```

In lldb: `p this`, then `-c 'this == 0x16fdff02c'`. RAN (I recorded the address with a one-shot breakpoint and printed it.) The address changes each run, so redo this step each session.

By thread. `break LOC thread N` limits a breakpoint to one thread. See the manual's *Thread-Specific Breakpoints* section; I did not read that page.

By string. `$_streq(name, "left")` compares text. Do not use `==` on `char*`.

Other useful forms

Goal	gdb
Breakpoint on every function matching a pattern	`rbreak ^Link`
Run to a line without a permanent breakpoint	`until 120`, `advance 120`
Save and reload your breakpoints	`save breakpoints probes.gdb`, later `source probes.gdb`
Breakpoint on a location not loaded yet	`set breakpoint pending on`

`save breakpoints` is how a one-shot setup becomes a repeatable one (chapter 14). The manual warns that watchpoints on local variables "may fail to be recreated."

Check yourself

- What is the difference between an ignore count and a counter variable?
- How do you read how many times a breakpoint was reached?
- You have 100 objects of one class. How do you stop only for one?

CHAPTER 11

Watchpoints

A breakpoint follows code. A watchpoint follows data. When you know *what* is wrong but not *where* it was made wrong, a watchpoint finds the writer.

Just enough memory

You said your C knowledge stops at the edge of memory, and that you only need what you can see in an editor. One page is enough for now.

- A **variable** is a named box. It lives at an **address**.
- A **pointer** is a box that holds another box's address. `*p` means "the box `p` points at."
- `&x` means "the address of `x`."
- An **object** is one set of boxes. `this` is the address of the one you are inside. Two objects of one class are two sets of boxes at two addresses.

That is why you can ask three different questions about a pointer `p`:

You type	You are watching	It stops when...
`watch p`	the **pointer box**	`p` is reassigned to point elsewhere
`watch *p`	the **pointee box**, as `p` currently points	the target changes (and GDB re-evaluates if `p` moves)
`watch -l *p`	the **memory location** it points at *right now*	that exact location changes, even after `p` changes

If a pointer value is changing and affecting the system but you do not know where, start with `watch p`. Then, once you have the address you care about, use `watch -l` on it. You do not need to read assembly for any of this.

The three kinds

Command	Stops when...	Notes
`watch EXPR`	the value is **written** (and changes)	the common one
`rwatch EXPR`	the value is **read**	hardware only
`awatch EXPR`	the value is **read or written**	hardware only

Add `-l` (or `-location`) to watch the **memory** the expression refers to, not the variable's scope. The manual: "The `-location` argument tells GDB to instead watch the memory referred to by expr." MANUAL

Rules you must know

From the GDB manual unless noted. MANUAL

- **Hardware first.** "GDB sets a *hardware watchpoint* if possible." Hardware watchpoints do not slow the program. Software watchpoints single-step and "report the change in value at the next *statement*, not the instruction."
- **Limited supply.** The manual gives no number. "If you set too many hardware watchpoints, GDB might be unable to insert all of them when you resume." The warning reads `Could not insert watchpoint`. On Cortex-M parts, the comparators live in the DWT unit. The Memfault example dump shows 4 on one nRF52840. Read `DWT_CTRL.NUMCOMP` for yours.
- **Scope.** A watchpoint on a local is deleted when the function returns: "GDB automatically deletes watchpoints that watch local (automatic) variables … when they go out of scope." Use `-l` to keep it.
- **Same value, no stop.** If a write stores the value that is already there, plain `watch` does not stop. The Memfault article says GDB "will suppress the debug event generated and auto-resume the processor." To see *every* write, use `awatch` or a breakpoint.
- **The stop is after the write.** The program has already moved on. The Memfault article says that on its target the PC "has already advanced" and the halt can be a couple of instructions late. In source terms: you stop on the line *after* the assignment.
- **Alignment (embedded).** "it is only possible to watch ranges of addresses that are aligned on the size being watched."
- **Conditions work.** `watch EXPR if COND`.
- **Constants cannot be watched.** `watch 0x600850` fails with "Cannot watch constant value." Use `watch *(int *) 0x600850`.

Recipes

1. Who writes this? (LOG + TRAP) RAN

Stop in a frame where the object is visible, put a location watchpoint on the field, and log each write without stopping. This is the recipe for **"which instance, which line"**.

```
break Gateway::run
run --max 400
watch -l monitor_.state_            # gdb: put the log in a commands list (lab/gdb/02-
log-state-writes.gdb)
```

lldb, with a Python callback. Real output:

```
state_ written -> Up        after line 115   via LinkMonitor::on_heartbeat() <-
LinkMonitor::on_event
state_ written -> Degraded  after line 97    via LinkMonitor::on_tick() <-
LinkMonitor::on_event
state_ written -> Down      after line 102   via LinkMonitor::on_tick() <-
LinkMonitor::on_event
state_ written -> Up        after line 115   via LinkMonitor::on_heartbeat() <-
LinkMonitor::on_event
state_ written -> Degraded  after line 97    via LinkMonitor::on_tick() <-
LinkMonitor::on_event
state_ written -> Down      after line 102   via LinkMonitor::on_tick() <-
LinkMonitor::on_event
state_ written -> Up        after line 115   via LinkMonitor::on_heartbeat() <-
LinkMonitor::on_event
```

Seven writes, three distinct places. This is a **complete list of where the state changes**, at runtime. (Notice one place that never appears. Chapter 23 explains why that matters.)

2. Only the bad values (TRAP with a condition) RAN

```
watchpoint set variable this->monitor_.missed_
watchpoint modify -c 'missed_ == 7'
```

It stopped in `on_tick` right after `++missed_` made it 7, at t=26. In gdb: `watch -l monitor_.missed_ if missed_ == 7`. MANUAL Note that in a condition the names are looked up in the frame where the stop happens, so write `missed_`, not `this->monitor_.missed_`.

3. Who reads this? (`rwatch`)

`rwatch -l monitor_.state_` stops when anything *reads* `state_`. That finds the consumers: the code that reacts to the value. Expect a lot of hits in a loop. Combine with a condition or an arm. MANUAL

4. Only one instance

`watch -l` on one object's field already watches one location. If you pick the field through `this` while stopped inside the object you want, you have chosen that instance. To scope a *breakpoint* to one instance, use `this == $target` (chapter 10).

5. Arm, then watch (cuts the noise)

A busy field changes thousands of times. Keep the watchpoint disabled until a trigger event, then switch it on. MANUAL

```
watch -l monitor_.missed_
disable 2                                  # keep watchpoint 2 off
break LinkMonitor::announce if state_ == LinkState::Degraded
commands
  silent
  enable 2                                 # armed: now watch missed_
  continue
end
```

The full script is `lab/gdb/06-arm-then-watch.gdb`.

6. The program ends and the watchpoint fires anyway

I hit this. With a watchpoint on a variable inside `main`'s objects, lldb kept resuming from watchpoint hits after the program had printed its summary and was shutting down (my reading: the stack memory was being reused). RAN Put a breakpoint near the end of the run and `delete` the watchpoint there. The lab scripts do this at `Gateway::summary`.

Embedded notes

- Data watchpoints are implemented in silicon, so the count is small and chip-specific.
- The stop happens just *after* the access. Read the line above the stop line.
- Writes of an identical value may not stop. If a peripheral register is rewritten with the same value, use `awatch`.
- Memfault's article "Faster Debugging with Watchpoints" walks through Cortex-M specifics with an nRF52840.

Check yourself

- What is the difference between `watch p`, `watch *p`, and `watch -l *p`?
- Why do you stop on the line *after* the write?
- How do you keep a watchpoint from disappearing when the function returns?

CHAPTER 12

Seeing state the way your editor shows it

You said you only want to see what you would write or read in a text editor: arguments, classes, variables. Good. That is all this chapter covers. No memory dumps, no assembly.

"Does `info locals` show all the variables of my object?"

You asked this directly: for an initialized class, how do I see the state of all its variables in a given frame, or will `info locals` only show what is visible in `list`?

No, it does not show the object's members. `info locals` and `info args` show the **local variables and arguments of the selected frame**. The GDB manual describes `info locals` as "all variables (declared either static or automatic) accessible at the point of execution of the selected frame." It does not claim that the members of `this` are included, and in practice they are not. To see the whole object, print the object.

Here it is on the practice program. Stopped inside `LinkMonitor::on_event`: **RAN**

```
(lldb) frame variable
(LinkMonitor *) this = 0x000000016fdff02c
(const Event &) e = 0x000000016fdfefd0 (type = Heartbeat, tick = 26, payload = 0)

(lldb) p *this
(LinkMonitor)  (state_ = Degraded, missed_ = 7, good_ = 1, now_ = 26, data_seen_ = 8)

(lldb) p e
(const Event &) 0x000000016fdfefd0 (type = Heartbeat, tick = 26, payload = 0)
```

`frame variable` showed the two things the *frame* owns: the pointer `this` and the argument `e`. `p *this` showed every member of the object. In gdb the same two steps are `info args` and `print *this`. **MANUAL**

> **RULE** **To see an object: `print *this` inside a method, `print obj` where the object is in scope, `print *ptr` through a pointer.** The frame's locals are one thing; the object's members are another.

What do I want to see?

I want to see…	gdb	lldb
Arguments of this call	`info args`	`frame variable` **RAN**
Local variables	`info locals`	`frame variable` **RAN**
All members of the object I am in	`print *this`	`p *this` **RAN**
One member	`print this->missed_` or just `print missed_`	`p missed_` **RAN**
An object through a pointer	`print *ptr`	`p *ptr`
An argument passed by reference	`print e`	`p e` **RAN**
What a call returned	`finish` (prints “Value returned”)	`finish`
Locals of **every** frame in the stack	`bt full`	`bt`, then `frame select N`
The definition of a type	`ptype LinkMonitor`	`type lookup LinkMonitor`
A value that updates at every stop	`display missed_` (and `undisplay N`)	`display missed_` (untested)

Two of these are worth pausing on.

`finish`. It runs until the current function returns and prints what it returned. “What did this call give back?” is the *out* half of “what went in, what is in here, what came out.”

`display`. It prints an expression every time the program stops. Put the three or four variables from your card on `display` and every stop becomes a status line.

Moving between frames without moving the program

A backtrace is a stack of frames. You can **look** at any of them without running anything.

Goal	gdb	lldb
Show the stack	`bt`	`bt` **RAN**
Look at the caller	`up` (`up 2`)	`up` **RAN**
Back down	`down`	`down`
Jump to frame N	`frame 3`	`frame select 3`

`up` does not move execution. It changes which frame `print`, `info locals`, and `info args` refer to. This is how you answer “what did the caller pass in?” For example, `up` from `on_event` lands in `Gateway::dispatch`, where `frame variable` shows the Gateway’s own

`this` and the same `e`. RAN

Making the output readable

Setting	Effect
`set print pretty on`	One member per line for nested structs. Default is off.
`set print object on`	For polymorphic objects, show the actual derived type instead of the declared one. Default is off.
`set print elements unlimited`	Show whole arrays and strings. The default limit is 200 elements.

Put them in a startup file so they are always on (chapter 14).

Containers and library types

`print vec` for a `std::vector` should show the elements if pretty-printers for the standard library are installed. Linux distributions normally ship them with GDB, but I have not verified this for your server. If you see raw internal pointers, that is the sign they are not loaded. lldb includes built-in formatters for the C++ standard library. UNTESTED

When a value says `<optimized out>`

The program was built with optimization. Rebuild with `-O0 -g` (chapter 4). Debugging an optimized build is possible but you spend your attention fighting the tool.

Check yourself

- Which command shows the members of `this`? Which shows the arguments?
- You are stopped in a function and you want to know what its caller passed. What do you type?
- What does `finish` give you that `next` does not?

CHAPTER 13

Rewind, snapshots, and crashes

The state is bad *now*, but the cause was earlier. There are three ways to get back to it: run again with a better probe, record and rewind, or examine a snapshot taken at the moment of failure.

Option 1: run it again (the cheap rewind)

This is the one you will use ninety percent of the time. If step 1 made the run **repeatable**, you can simply restart and follow the data from the beginning to the point where it goes bad. In your words: note the thing that is off, restart, and follow that variable from initialization to the bad state.

1. A FILTER probe finds the first bad moment (say, tick 26, `missed_ = 7`).
2. Note the **tick or count** at which it happened.
3. Restart. Put a TRAP or LOG on the variable, and an **ignore count** or **arm** so you only look near the bad moment.

This is why determinism matters so much. A bug you can replay exactly is a bug you can bisect.

Option 2: record and replay

When you cannot predict where to look until after you have seen the failure, record the whole run and move backward through it.

rr records a Linux run and replays it deterministically inside gdb. SEC

```
rr record ./prog args        # run once, recorded
rr replay                    # gdb, connected to the recording
```

Under rr, "gdb's `reverse-continue`, `reverse-step`, `reverse-next`, and `reverse-finish` commands all work." The pattern for *who wrote this bad value* is the one from the rr wiki: MANUAL

```
(gdb) break the_line_where_it_is_wrong
(gdb) continue                    # arrive at the bad state
(gdb) watch -l mRect.width        # watch that location
(gdb) reverse-continue            # run backward to the write that set it
```

The wiki says `watch -l` “is important”, for the reason in chapter 11: it watches the memory, not a scope.

Limits from the rr site: it “requires a reasonably modern x86 CPU or certain ARM CPUs (Apple M1+)”, “emulates a single-core machine”, and the recorded executable must not change before replay. Single-core emulation can hide or change the behavior of timing-dependent bugs. I do not know whether your Linux server exposes what rr needs (virtual machines sometimes do not), so test it before you plan around it.

gdb’s own `record` gives reverse commands without rr: MANUAL

```
break main
run
record
continue
reverse-continue
```

The manual says “not all target environments can support reverse execution”, and that reverse-next may fail “if the code is too heavily optimized.” Recording in gdb is slow, so use it on short windows (start `record` shortly before the event you care about).

> NOTE **Embedded targets.** Reverse execution needs a target that supports it. On most microcontrollers over a hardware probe it is not available. Use option 1 (repeatable runs with probes) or option 3.

Option 3: a snapshot at the moment of failure (core dumps)

A **core dump** is the memory and thread state of a process written to a file when it crashes. You cannot run it. You can inspect it: stack of every thread, every variable in every frame. UNTESTED

```
ulimit -c unlimited            # allow core files in this shell (Linux)
./prog                         # crashes, writes a core file
gdb ./prog core                # load program and core together
(gdb) bt                       # where did it die?
(gdb) frame 2                  # select a frame
(gdb) print *this              # look at that object
(gdb) info threads
(gdb) thread apply all bt      # the stack of every thread
```

Where the core file goes is a system setting (`cat /proc/sys/kernel/core_pattern`). You need the **same binary, built with symbols**, that produced the core.

Your notes from the GDB Deep Dive talk describe the embedded version of this: after release, a device that crashes in the field sends a core dump, and each RTOS thread (main thread, system workqueue, service threads) has its own stack to inspect. You said you will not be doing that while training, and may later. All of the *reading* skills in chapter 12 carry over.

Crashes: the first move

For a crash (a signal such as SIGSEGV), the first move is simple.

1. Run under the debugger. It stops at the signal. Run `bt`.
2. The crashing line is rarely the cause. Walk **up** the stack (`up`) and look at which pointer or index is bad.
3. Ask: *where did that value come from?* That is a TRAP question. Restart and `watch -l` it, or use rr and `reverse-continue`.

`catch signal` stops on a signal without having to wait for the crash, and `catch throw` stops where a C++ exception is thrown rather than where it is caught. MANUAL

Compilers can also help find memory errors before they become crashes. AddressSanitizer (`-fsanitize=address`) is the standard first tool for memory bugs in C and C++. UNTESTED

Which option when?

Situation	Use
You can reproduce it every time	Option 1: run again with a probe
You see it, but cannot guess where to probe	Option 2: record, then rewind
It only happens in the field, or once in a thousand runs	Option 3 (core dump), or record in a loop until it fails
It crashes	Debugger stops at the signal, then walk up and trace the bad value back

Check yourself

- What property of step 1 makes option 1 work?
- Why is `watch -l` needed before `reverse-continue`?
- What can you do with a core dump that you cannot do with a log?

CHAPTER 14

Make probes repeatable

A probe you type by hand is gone when the session ends. A probe in a file can be run again, improved, shared, and attached to a ticket. That is the difference between "I debugged it" and a repeatable result.

Why this matters

In the debugging study, one expert said breakpoints are "like a one-shot. Ideally I wouldn't like them to be, but I just set them anew." The authors also observed that "debugging is an internal thought process not usually shared." (Beller et al., 2018.) Saving your probes fixes both. You stop re-creating them, and someone else can read exactly what you asked the program.

Four levels, cheapest first

Level 1: save what you typed

```
save breakpoints probes.gdb          # in a session                    [MANUAL]
source probes.gdb                    # in a later session
```

The file is plain gdb commands, so you can edit it. The manual warns that watchpoints on local variables "may fail to be recreated."

Level 2: a script you run in one command

Write the probe as a file and run gdb non-interactively. MANUAL

```
gdb -q -batch -x gdb/01-invariant.gdb ./gateway
```

- `-x FILE` runs the commands in the file as if you had typed `source FILE`.
- `-batch` "exit with status 0 after processing all the command files", and turns off paging and confirmations.
- `-ex 'COMMAND'` runs a single command and can be repeated.
- "An error in any command terminates execution of the command file", so a typo stops the run instead of quietly doing nothing.

lldb: `lldb -b -s lldb/01-invariant.lldb ./gateway`. **RAN** Every probe in the book's lab is a file you can run this way. Start with `set pagination off` in gdb scripts so output is not held at a prompt.

Do not depend on a project-local `.gdbinit`. GDB reads a `.gdbinit` in the current directory last, but it will not blindly run files from untrusted places (the `auto-load safe-path` setting), so it may be refused and print a warning. Passing the script with `-x` always works.

Level 3: Python for conditions that need real code

When a condition is too awkward for one expression, write it as a Python class. A `gdb.Breakpoint` subclass overrides `stop()`: return `True` to stop, `False` to keep running. **MANUAL**

```
import gdb

class StaleMissed(gdb.Breakpoint):
    def stop(self):
        etype = str(gdb.parse_and_eval("e.type"))
        missed = int(gdb.parse_and_eval("missed_"))
        return etype.endswith("Heartbeat") and missed != 0

StaleMissed("gateway.cpp:81")
```

Rules from the manual: inside `stop()` you must not step, change the frame, or add or delete breakpoints. Reading and printing are what it is for. The lldb equivalent is a callback function that returns `False` or `True`, which you have already seen in `lab/lldb/probes.py`. **RAN**

This is also where "gdb Python API for testing and team workflows" starts to pay off. A probe file that sits next to a test is a regression check you can run against a live process.

Level 4: a nicer screen

gdb-dashboard is "a standalone `.gdbinit` file written using the Python API that enables a modular interface showing relevant information about the program being debugged." Per its README, it installs by downloading one file, and "the dashboard will appear automatically every time the inferior program stops." It needs a gdb built with Python.

> **TRAP** **Check before you install anything on a work machine.** gdb-dashboard is a file downloaded from the internet that gdb then executes. Ask your team or IT whether it is allowed, and if so use a reviewed, pinned copy. To find out whether your gdb has Python: `gdb -batch -ex 'python print(1)'`. To find out whether it is already set up: look for `~/.gdbinit` and ask what your team uses.

VS Code shows the same information in panes (Variables, Call Stack, Watch) and lets you set breakpoints in the file you are editing. Its breakpoint types include: SEC

- **Conditional**: an expression, a hit count, or "wait for breakpoint". The docs note that hit-count syntax "varies by debugger."
- **Logpoints**: "writes a message to the Debug Console without pausing execution." Write text with expressions in curly braces, such as `heartbeat t={e.tick}`. Available only if the debugger extension implements them.
- **Data breakpoints**: pause when a variable changes. The docs say the options "depend on the debugger" and do not say which debuggers support them. Try it with your setup. If the menu is not offered, use a raw gdb command.
- **Raw commands**: in the Debug Console, the C/C++ extension lets you run gdb commands with the `-exec` prefix (for example `-exec watch -l missed_`). The docs warn this is "untested and might crash VS Code in some cases."

Appendix A has a step-by-step setup for VS Code over SSH, which you said is on your backlog.

Conventions that keep a probe library useful

- **One file per probe.** Name it by task and behavior: `TKT-1234-stale-missed.gdb`.
- **Put the card reference and the prediction in the header comment.** The next person needs the *question*, not only the commands.
- **Use function names over line numbers when you can** (`break LinkMonitor::on_event`). Line numbers drift when the file changes. If you must use a line, say which line it is in a comment.
- **Delete watchpoints near the end of the run** so they cannot fire during shutdown.
- **Keep them in the repository** (for example `tools/probes/`) or in your own notes, whichever your team prefers. Commit them with the fix that they proved.

Check yourself

- What does `-batch` change about how gdb runs?
- When would you write a Python probe instead of a conditional expression?
- Why is a probe file better than a screenshot in a ticket?

CHAPTER 15

Step 5 · Pin it

You often know the module or component you are in, then start guessing which instance of a variable to change. Pinning ends the guessing. The goal is one sentence that names the exact line and the exact instance.

What “pinned” looks like

> *`missed_` stays at 7 after the Heartbeat at t=26, because `on_heartbeat` does not reset it when the state is Degraded.*

That sentence names **the data** (`missed_`), **the instance** (the one `LinkMonitor`, seen at tick 26), **the place** (`on_heartbeat`, the Degraded branch), **the condition** (state is Degraded), and **the reason** (it is not reset). You can test it. That makes it a hypothesis, not a hunch.

Zeller’s chain, as a search

Every failure you can see is the end of a chain. Each link is a **state**.

```
defect (a line of code)  ->  infection (the first bad state)  ->  ...  ->  failure
(what you saw)
```

Your observed symptom (the link went Down) is the *failure*. Somewhere upstream is the *first bad state*, the **infection**. Everything between the infection and the failure is the infection spreading. Your job is to walk back until you reach the first state that is wrong, because the cause is in the code that produced exactly that state.

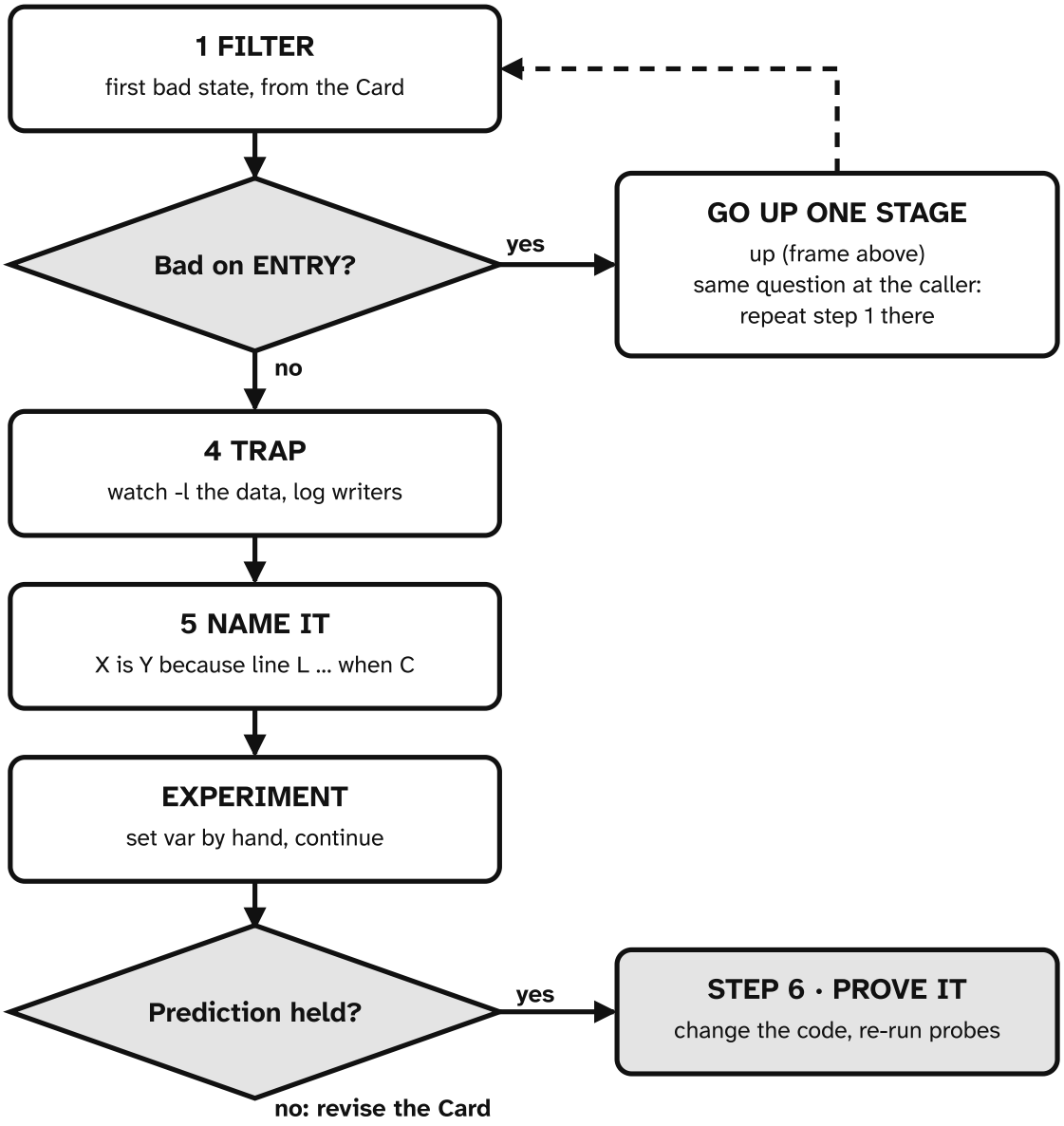


Figure 15.1. Pin it. Walk back along the infection chain one stage at a time until the data is good on entry and bad on exit. Then the bad write is inside that one function.

The five moves (for a bug)

1. Find the first bad observation. Use the FILTER probe from your card. It stops at the earliest moment the rule is broken. Not the visible symptom, which comes later.

2. Is the data bad on entry, or made bad inside? At the stop, look at the arguments and the object (`info args`, `print *this`). Then look at the caller (`up`, `print`). If the data was already bad when this function started, the infection is earlier. If it was good on entry and bad now, the bad write is inside.

3. If bad on entry, go up one stage and repeat move 1. Same question, one stage earlier. This is bisection along the data path (Agans's *divide and conquer*; the SRE book's *bisection* of communication paths).

4. If made bad inside, find the writer. Read the function with the card beside it, and check which branch does not do what P and Q say. Or TRAP: put a `watch -l` on the data and log each write with its line (chapter 11). Compare the writes you see with the writes the Card says should happen.

5. Name it. Write the one sentence. Then test it.

> **RULE** **Test the sentence before you edit any code.** Stop at the first bad state, change the value by hand, continue, and see whether the symptom goes away. That is the experiment in Zeller's loop: hypothesis, prediction, test.

```
(lldb) expression missed_ = 0
(lldb) continue
[t= 27] Degraded -> Up       (missed=0 good=2)        <- the behavior the requirements
ask for
```

RAN Changing one value by hand turned `Degraded -> Down` into `Degraded -> Up`. The sentence survived the experiment. Now you are allowed to change the code (step 6). If it had not, you would revise the card.

Which instance? (the "100 instances" problem)

Your original problem: a variable that shows up a hundred times, in several methods, through pointers, and in and out of a messaging system. Use these rules in order.

1. **Do not pick by name. Pick by state.** Let the FILTER probe stop *in* the instance that is wrong. `this` at that stop *is* your instance. (`print this` gives the address.)
2. **Record it.** `set $target = this` in gdb. In lldb, copy the address from `p this`. Now you can say "this one."
3. **Scope the next probe to it.** `condition N this == $target`, or `watch -l` on that object's field (which fixes one memory location).
4. **If data crosses a message boundary, follow the value, not the variable.** Tag it: a sequence number, an id, or any distinctive field. Put a LOG probe on the *send* and on the *receive*, filtered on the tag. The two log lines show the same message on both sides.
5. **If the instance is rebuilt or copied, re-find it.** Addresses change. A stable field (`id`, `name`) plus `$_streq` finds it again.

TRAP **Changing “the variable that looks right.”** A name that appears in ten places means ten candidate writes. Choosing one because the name matches is the guess this chapter exists to remove. Let the *first bad state* choose for you.

Pinning for a feature: find every change point

For a feature there is no bad state. You are looking for **where the new behavior has to go** and **everywhere that must agree with it**. The same probes answer it.

1. From the card, the feature is a *before and after*: “when the link returns to Up, report how long the outage lasted.”
2. List the **writers and readers** of the data involved. Use the writer log from chapter 11.
3. For each writer, ask: is this a place my feature has to hook into?
4. Mark **paths the log did not show**. They are places the code *can* reach that your test run did not.

In the practice program, the writer log shows **three** places where `state_` is written during the run, but the code has **four**. One path, `Degraded` to `Up`, never ran, because the bug prevented it. A feature added at “the place the link becomes Up” would have been added to only one of two places. Chapter 23 walks through exactly this.

RULE **Runtime shows what ran. Reading shows what exists.** Use both. Compare the list of writers you *saw* with the list in the code. Every difference is a path you must reason about or exercise.

How long should this take?

About 5 to 10 percent of the task (chapter 20). Past 1.5 times that with no sentence, stop probing blindly. Re-check the card: is the **expected** value right, and is this really the data that matters? Or ask, with your best hypothesis in hand.

Check yourself

- What is the difference between the failure and the infection?
- How do you know whether to go up one stage or look inside this function, and why test the sentence before you edit the code?

PART 5

The three families

Every task is one of three kinds: understand, fix, or build. The seven steps are the same. The first probe and the time split change.

CHAPTER 16

Intake: classify the task in five minutes

When you are handed work, you want to allocate time based on a process. That starts with naming what kind of work it is. Five minutes of intake decides your first probe, your budget, and when you ask for help.

Three questions

1. **What is being asked?** To *explain* something (Family A), to make something *stop being wrong* (B), or to make it do something *new* (C).
2. **How sure am I of the expected behavior?** Can you write the EXPECTED line of the card, with a source? If not, your first job is step 2.
3. **How many unknowns?** Start a card, fill what you can, and count the `?` marks.

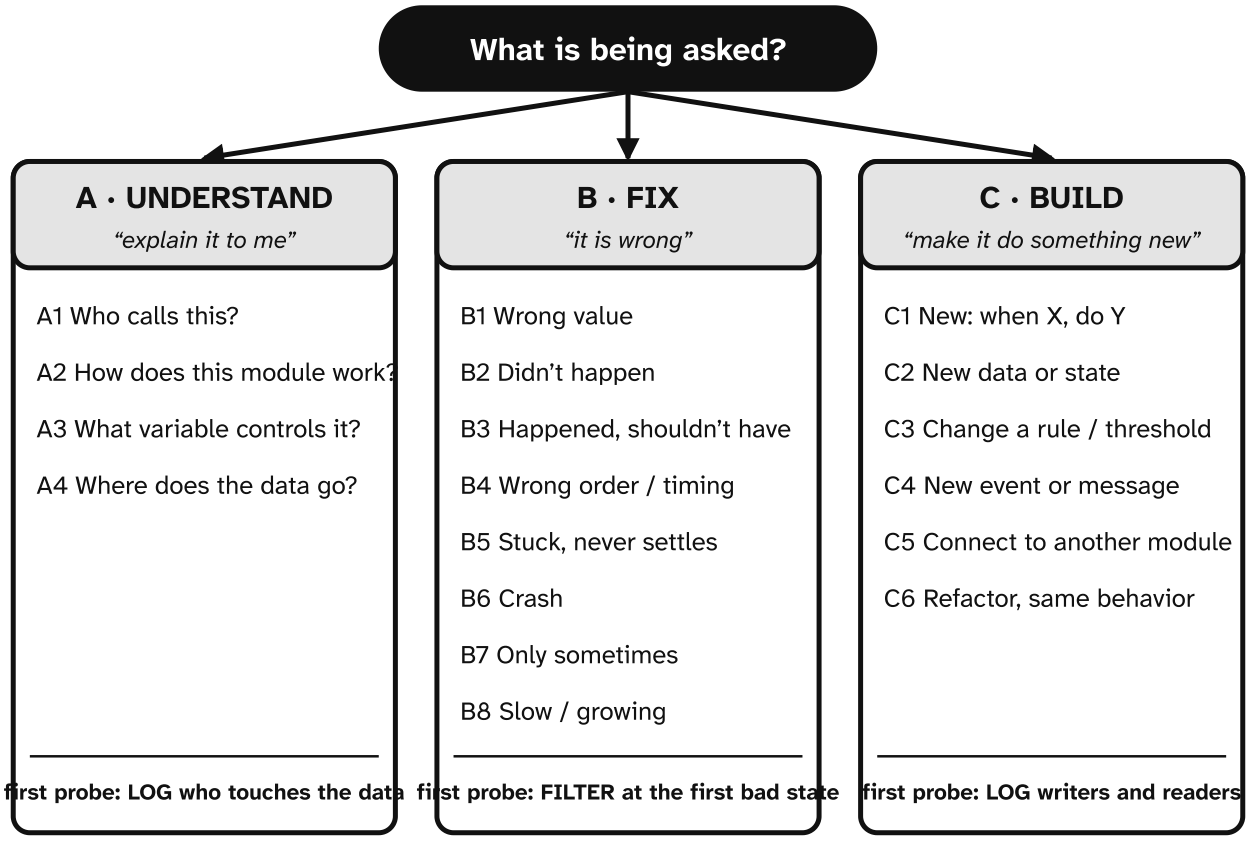


Figure 16.1. Three families, eighteen types. Mixed task? Do the understanding part first.

The eighteen types

Find the closest row. If two fit, you have a mixed task (see below).

Type	It sounds like...
A1 Who calls this?	“Where is this module used? Is that the only place?”
A2 How does this module work?	“I don’t know how this code works.”
A3 What controls it?	“Which variable do I change to get this behavior?”
A4 Where does the data go?	“What happens to this value between input and output?”
B1 Wrong value	“It shows 7 but should show 0.”
B2 Didn’t happen	“The alarm never fires.” “Nothing is logged.”
B3 Happened, shouldn’t	“It sent the message twice.” “It dropped the link while heartbeats arrive.”
B4 Wrong order or timing	“A should come before B.” “Works alone, fails with two clients.”

Type	It sounds like...
B5 Stuck	“It hangs.” “The loop never exits.” “It never recovers.”
B6 Crash	“Segmentation fault.” “Core dumped.”
B7 Only sometimes	“One run in fifty fails.”
B8 Slow or growing	“It gets slower.” “Memory keeps climbing.”
C1 New behavior on a condition	“When the link recovers, log the outage length.”
C2 New data or state	“Track how many times the link flaps.”
C3 Change a rule	“Degrade after 3 missed ticks, not 4.”
C4 New event or message	“Handle a new `Reset` message.”
C5 Connect to another module	“Report status to the logger service.”
C6 Refactor	“Clean this up, same behavior.”

The Intake Line

Write one line at the top of the card. It is the whole plan in a sentence.

```
INTAKE   B3 · first probe: FILTER at end of on_event · 4 h · 3 unknowns · check-in at 2 h
```

- **Type** from the table.
- **First probe** from the playbook (chapters 17 to 19).
- **Time** you will spend before you ask someone. Chapter 20 splits it.
- **Unknowns** from the card.
- **Check-in**: when you will tell your lead where you are.

Unknown count: a rule of thumb

This is a heuristic from this book, not a measured result.

Unknowns on the card	What to do
0 to 2	Go.
3 to 5	Add extra time to step 2. Your first job is to turn unknowns into names (chapter 8).
6 or more	The task is not yet defined. Split it, or get the expected behavior clarified before you probe.

Mixed tasks

Most real tasks are a mix. Handle them in this order:

- **A bug fix usually contains an A.** “Why does X happen” often needs A2 (how does this module work) first. Do the A, then the B.
- **A feature often contains an A3 or A1.** To add behavior you must find what controls the related behavior and where it is called.
- **Do the smaller family first.** Understanding tasks are cheap and make the other two faster.
- **Write one card per behavior.** If you have two behaviors, you have two cards, even in one ticket.

TRY IT Pick three tasks from your history: one bug, one feature, one “figure out how this works”. Find their types. Which row had you spent the most time on without realizing what kind of task it was?

Check yourself

- What are the three questions at intake?
- When do you stop and clarify instead of probing?
- What goes on the Intake Line?

CHAPTER 17

Family A · Understand tasks

“I don’t know how this code works.” “I don’t know where this is called.” “What variable controls this?” These are your first three scenarios. The output is not a change to the code. It is a map you can trust.

What every A task produces: a short written summary, one diagram (state by event table, call list, or data path), and a Behavior Card whose DEFINE lines you can *predict* against. You are done when you can pass the Confidence Gate for the thing you were asked to understand.

A1 · Who calls this? Is that the only place?

LOOKS LIKE “Where is `LinkMonitor::on_event` used? Does anything else call it, under conditions I have not considered?”

STATE SIGNATURE A function F is reached from some context. You want the set of contexts and the condition that selects each.

FIRST PROBE LOG the callers: `break F`, commands `silent`, `bt 2`, `continue`. Run a full scenario and tally with `sort | uniq -c`. Add a static search (`rg "F("`, *Find All References*).

THEN For each caller, find the condition that leads to it (read the caller, or FILTER at the call site). Look for **rare callers**: filter by caller (`$_caller_matches`) or break at a call-site line.

DONE WHEN You have a list of call sites, each with its condition, and each marked *seen at runtime* or *only seen in code*.

TRAP Runtime shows only what ran. A caller that your scenario never triggers stays invisible. Compare against the static list.

A2 · How does this module work?

LOOKS LIKE "I just got this code. I don't know what it does."

STATE SIGNATURE The module holds state and reacts to events. The behavior is its state-by-event table.

FIRST PROBE TRAP and LOG the writes to the module's state (chapter 11). In parallel, log which events arrive and in what state.

THEN Build the table of what *actually* happened (state by event). Put it beside the table you built from the requirements (chapter 5). Every cell that differs is a question. Every cell that never appeared is a path you have not exercised.

DONE WHEN You can fill the state-by-event table and predict the next transition before it happens.

TRAP Reading every function in order. Start from the state variable, not from `main`.

A3 · What variable controls this behavior?

LOOKS LIKE "I need to adjust something but I do not know which variable affects it." (And the name appears a hundred times.)

STATE SIGNATURE An output O changes (or fails to). O is computed from inputs.

FIRST PROBE TRAP the *output*: `watch -l O`, with a backtrace log. Each write shows the line that computed it.

THEN Walk the dependency chain backward. At the writing line, read the variables it uses (the inputs). Repeat on each input until you reach a configuration value, a constant, or an external input. **Count the distinct variables on the way.** Test each candidate by changing it by hand (`set var`) and watching whether O responds.

RATE THE COMPLEXITY (this book's heuristic): 1 to 2 variables, simple. 3 to 5, moderate. More than 5, high. **Can I change one variable in one method?** Yes if one `set var` at one place gives the behavior you want.

DONE WHEN You can name the variable, the line that reads it, and the experiment that shows it controls O.

TRAP Picking by name. Pick by *effect*.

A4 · Where does this data go?

LOOKS LIKE "What happens to this value between input and output?" Possibly across a messaging system.

STATE SIGNATURE One value travels through several stages.

FIRST PROBE **Tag the value.** Feed in a distinctive input (a sequence number, an easily recognized value). FILTER or LOG at each stage boundary on that tag.

THEN The sequence of log lines *is* the data path. Draw it. Mark every stage where the value is copied, changed, or dropped.

DONE WHEN You have a data path diagram where every arrow is backed by a log line.

TRAP Following the variable *name*. Names change across stages; the tagged value does not.

The Understand deliverable

A one-page summary you could hand to the next person:

```
MODULE:     LinkMonitor
PURPOSE:    Decide link health so data flows only on a healthy link.
STATE:      state_ (Down/Up/Degraded), missed_, good_
EVENTS:     Tick, Heartbeat, Data      (table attached)
CALLED BY:  Gateway::dispatch (every event); Gateway::flush_burst (rare: Data with
payload >= 99)
WRITERS:    on_tick (2 sites), on_heartbeat (2 sites)
UNKNOWNS:   ...
```

Chapter 22 builds this one for real, for the practice program.

Check yourself

- For A1, why is a runtime log not enough on its own?
- For A3, how does counting variables help you decide whether a one-line change is realistic?

CHAPTER 18

Family B · Fix tasks

A bug is behavior: the system will not do something it should, or does the opposite of what you want. Describe the behavior as a state, find the *first* bad state, and everything else follows.

The standard fix loop

Every B type uses the same loop, with a different first probe.

1. **Run it.** Reproduce. Save the run record.
2. **Card.** Expected, observed, Declare, Define (P, Q, I).
3. **First probe** from the type (below). **Predict first.**
4. **Pin it.** First bad state, then the one sentence (chapter 15).
5. **Experiment.** Change a value by hand (`set var`) and see if the symptom changes.
6. **Change the code.** The smallest change that makes the sentence false.
7. **Prove it.** Re-run the same probes; they must be silent. Prove they could fire (chapter 24).

The eight types

B1 · Wrong value

LOOKS LIKE "It shows 7 but should show 0."

STATE SIGNATURE Variable `v` has a value that violates Q at a known place.

FIRST PROBE FILTER at that place: `break PLACE if v != EXPECTED` (or the rule as an expression).

THEN `up`, compare arguments to the wrong value. Bad on entry: go up one stage. Made inside: TRAP the writes to `v`.

DONE WHEN You can name the line that produces the first wrong value.

TRAP Fixing where the wrong value is *shown* instead of where it is *made*.

B2 · Didn't happen ("why didn't it?")

LOOKS LIKE "The alarm never fires." "Nothing is logged." "The second message is never sent."

STATE SIGNATURE The action A never ran, so either its guard G was false or G was never evaluated.

FIRST PROBE Breakpoint on A. If it has **zero hits**, go up. Break at the **decision** that should lead to A, and log each operand of G.

THEN Find the first operand that is not what the rule says. If the decision is never reached, repeat one stage up the call chain.

DONE WHEN You can say which operand made G false, or which call never happened.

TRAP Putting the probe on the action. If it never runs, the probe never fires and tells you nothing. Probe the *decision*. (This is the Whyline's *why didn't* question, by hand.)

B3 · Happened but shouldn't have

LOOKS LIKE "It dropped the link while heartbeats were arriving." "It sent the message twice."

STATE SIGNATURE Action A ran while the rule says G_expected was false.

FIRST PROBE Breakpoint at A with `if !(G_expected)`. A `bt` shows *which path* led here.

THEN Compare the guard that let it through with G_expected. The guard is too loose, or reads stale data (`missed_` in the practice program).

DONE WHEN You can show the exact state in which A ran and say which part of the guard was wrong.

TRAP Stopping at the first path you find. There may be two callers. Check `bt` each time (chapter 8).

B4 · Wrong order, timing, or race

LOOKS LIKE "S2 happened before S1." "Fine with one client, fails with two."

STATE SIGNATURE Events or writes interleave in an order the rule forbids.

FIRST PROBE LOG both events with a sequence number or timestamp. Compare the order. For threads, log the thread id too.

THEN ARM: break at S1, enable the probe on S2. If it is a data race, TRAP the shared data with `watch -l`, and use `thread` scoping. Consider rr for replay.

DONE WHEN You have a log that shows the bad order, and the lock or ordering rule that should have prevented it.

TRAP Stopping changes timing. Prefer logs and counters to long stops. A race that disappears under the debugger is data.

B5 · Stuck, never settles

LOOKS LIKE "It hangs." "The loop never exits." "The link never recovers."

STATE SIGNATURE A state with no way out, or a progress variable that stops changing.

FIRST PROBE Let it hang. Interrupt (Ctrl-C). `bt` and `thread apply all bt`. Where is it spinning or waiting?

THEN Put a TRAP on the progress variable. If it never fires, nothing is progressing. Add a **tripwire**: break when an iteration counter passes a limit.

DONE WHEN You can name the state it is stuck in and the transition that should have left it.

TRAP Assuming a hang is a loop. It is often a wait (a lock, a read) with nothing to wake it. `bt` tells you which.

B6 · Crash

LOOKS LIKE "Segmentation fault." "Core dumped."

STATE SIGNATURE A signal at a line. The line is the *failure*, rarely the cause.

FIRST PROBE Run under the debugger; it stops at the signal. `bt`, then `up` to the frame with the bad pointer or index.

THEN Where did that value come from? That is a TRAP question: restart with `watch -l`, or use rr and `reverse-continue`. For a field crash, use the core dump (chapter 13).

DONE WHEN You can name where the bad value was created, not only where it was used.

TRAP Adding a null check at the crash site. That hides the infection and moves the failure.

B7 · Only sometimes

LOOKS LIKE "One run in fifty fails."

STATE SIGNATURE Behavior depends on something you are not controlling: timing, threads, uninitialized data, input.

FIRST PROBE Make it deterministic: fixed seed, single thread, fixed input (step 1). Run it in a loop until it fails and keep the *failing* log.

THEN Compare a failing log with a passing one (Agans: compare a bad run to a good one). `diff` them. The first difference is your lead. Record the failing run with rr if you can.

DONE WHEN You have a failing run you can replay, and you have found the first difference.

TRAP Debugging the run that passed.

B8 · Slow or growing

LOOKS LIKE “It gets slower.” “Memory keeps climbing.”

STATE SIGNATURE A resource grows or a rate drops over time.

FIRST PROBE **Measure first.** Pike’s rules: “You can’t tell where a program is going to spend its time” and “Don’t tune for speed until you’ve measured.” Use a profiler or a memory tool, not a guess (your `handbook/` chapter 09 covers profilers). In gdb, hit counts (`info breakpoints`) show which paths run unexpectedly often.

THEN Once the tool names a hot spot or a growing allocation, switch to B1 on it: what value or count should it have been?

DONE WHEN A measurement, not an opinion, shows the cause.

TRAP Optimizing the code that looks slow.

Two habits that apply to every B

RULE **Look for the first bad state, not the loudest symptom.** The failure is far downstream. The cause is the first state that violates the rule.

RULE **One change at a time.** If you change two things and the bug goes away, you do not know which one fixed it (Agans). Make the change, run the probes, then make the next.

Check yourself

- Which type is “the screen never refreshes”? Where does the probe go?
- Why is a null check at the crash site usually wrong?
- How do you debug a bug that only happens one run in fifty?

CHAPTER 19

Family C · Build tasks

Adding a feature is the same translation problem. You describe the new behavior in terms of what is already declared, defined, and implemented, then pin down the exact places where the change must go.

The standard build loop

1. **Run it.** See the *nearest existing behavior*. The feature will sit beside it.
2. **Understand it.** What should the new behavior look like, in things you can observe? Who relies on the old behavior?
3. **Card.** Write the feature as a **before and after**: "when P holds, afterwards Q holds." The card's EXPECTED line is the new behavior. Q′ (observed) is what happens *today* in the same situation.
4. **Probe.** LOG the writers and readers of the data the feature touches. Do not guess where the change goes.
5. **Pin it.** List every change point. Mark paths the log did not show.
6. **Build and prove.** Add the change. Show the new behavior in the run. Show the **old** behavior is unchanged.
7. **Log it.** Change points and the checks you used.

> **RULE** **A feature is complete when every path that can reach the new behavior has been considered.** Not when the one path you tested works.

The six types

C1 · New behavior on a condition (“when X, do Y”)

LOOKS LIKE “When the link recovers, log how long the outage lasted.”

STATE SIGNATURE A new action Y attached to a state change or event X.

FIRST PROBE TRAP the state that defines X (here `state_`) and LOG every writer with its line and call chain (chapter 11).

THEN Check **every** way X can become true. In the practice program, “the link becomes Up” happens in two places (from Down, from Degraded). Hooking only one is the classic miss.

DONE WHEN Y runs on each path in the list, in a run that exercises each.

TRAP Adding Y at the first path you find.

C2 · New data or state (“keep track of Z”)

LOOKS LIKE “Remember when the outage began.”

STATE SIGNATURE A new variable with a lifecycle: created, set, reset, read.

FIRST PROBE Declare Z on the card with its type, owner, and initial value. Find the events that **bracket** its life (where it must be set; where it must be read or cleared).

THEN Use the writer log of the *related* state to find the brackets. Choose a sentinel for “not set” (for example `-1`) and handle it where Z is read.

DONE WHEN Z is set on every path that starts the thing it measures, read on every path that ends it, and cleared so it cannot go stale.

TRAP Forgetting the reset. A value that is never cleared produces a wrong result on the *second* occurrence.

C3 · Change a rule or threshold

LOOKS LIKE “Degrade after 3 missed ticks instead of 4.”

STATE SIGNATURE The rule lives in a guard expression and often in a constant.

FIRST PROBE Find the guard (`missed_ >= kDegradeAfter`) and **all** uses of the constant (`rg kDegradeAfter`). Characterize today’s behavior first: save a log of the run (chapter 24’s *golden log*).

THEN Change the rule in one place. Re-run and diff the log: only the transitions you intended should move.

DONE WHEN The diff shows exactly the intended differences, and tests and docs agree.

TRAP Changing the constant but not the comment, the test, or the second place that duplicates the number.

C4 · New event or message type

LOOKS LIKE "Handle a new `Reset` event."

STATE SIGNATURE A new case in a dispatch point (a `switch`, a handler table, a registry).

FIRST PROBE Pick the most similar existing event and follow it end to end (type A4). Its path is your template.

THEN Add the new event along the same path. Turn on compiler warnings for missing `switch` cases (`-Wswitch`) so every place that must handle it is flagged.

DONE WHEN The new event travels the same stages as the template event, in a run you can show.

TRAP Handling it at the dispatch point but not in the state machine that reacts to it.

C5 · Connect to another module

LOOKS LIKE "Report status to the logger service."

STATE SIGNATURE Data crossing a boundary you do not own.

FIRST PROBE Write the **contract** on the card: what you send, what you expect back, what you assume. Then LOG traffic across the boundary (the send and the receive) to learn what the other side *actually* does.

THEN Stub the other side if you can, and test your half alone. Check the boundary in both directions.

DONE WHEN Your half works against the real thing *and* against a stub, and the assumptions are written down.

TRAP Believing the interface description over the log.

C6 · Refactor, same behavior

LOOKS LIKE "Clean up this class." "Move this into its own file."

STATE SIGNATURE The *state by event table* must not change.

FIRST PROBE Record a **golden log**: run the probes and the program before you touch anything and save the output.

THEN After each small step, run the same probes and `diff` against the golden log. Any difference is a behavior change, whether you meant it or not. This is Feathers's idea of a characterization test, done with probes.

DONE WHEN The diff is empty after the final step.

TRAP Changing behavior "while you are in there."

How to find where the change goes, in one table

After the writer and reader logs, fill this in. It is the output of **Pin it** for a build task.

Where (file:line)	What happens there	Seen in the run?	Needs the change?	Why

Rows with *seen = no* are the dangerous ones. Reason about them or make a run that reaches them.

Check yourself

- What does “complete” mean for a feature?
- In C1, why do you list every way X can become true?
- What does a golden log give you that a unit test does not?

CHAPTER 20

Time: budgets, checkpoints, and when to ask

When you get handed work you want to allocate time based on your process. Here is a way to do it: a starting split by family, checkpoints tied to the gates, and a rule for when to stop and ask.

> **NOTE** **Read this first.** The percentages in this chapter are **a proposal from this book, built from one research fact (comprehension takes more than half of developers' time on average) and common sense.** They are not a research result. Treat them as defaults to calibrate against your own journal.

The starting budgets



Figure 20.1. Starting budgets: percent of total task time per step. These are this book's proposal, not a research result. Calibrate with your journal.

Step	A · Understand	B · Fix	C · Build
1 Run it	10%	10%	5%
2 Understand it	30%	20%	25%

Step	A · Understand	B · Fix	C · Build
3 State it	20%	15%	15%
4 Probe it	25%	25%	15%
5 Pin it	5%	10%	10%
6 Prove it (includes the change)	5%	15%	25%
7 Log it	5%	5%	5%
Total	**100%**	**100%**	**100%**

Why steps 1 to 3 are heavy. Xia et al. measured about 58% of developer time as program comprehension in a field study of 78 professionals. A fix that skips understanding is not faster. It reaches the debugger without a condition to type. That is the reasoning behind giving steps 1 to 3 roughly 45 to 60 percent of the time.

Why C spends more on step 6. Building is mostly writing and checking the new behavior, plus confirming the old behavior survived.

How to use it

1. After intake (chapter 16), choose a total time **T**. Pick the time you will work before you ask someone, not a promise of the finish.
2. Multiply T by each percentage.
3. Write the plan at the top of the card. Look at the clock at each step boundary.

Here is a 4-hour task in each family.

Step	A · Understand	B · Fix	C · Build
1 Run it	24 min	24 min	12 min
2 Understand it	72 min	48 min	60 min
3 State it	48 min	36 min	36 min
4 Probe it	60 min	60 min	36 min
5 Pin it	12 min	24 min	24 min
6 Prove it	12 min	36 min	60 min
7 Log it	12 min	12 min	12 min
Total	**240**	**240**	**240**

For a 2-hour task, halve these numbers. For a full day (8 hours), double them. The shape stays the same.

Checkpoints tied to the gates

Gate	When it should pass	A	B	C
Gate 1: Confidence	after steps 1 and 2	by 40%	by 30%	by 30%
Gate 2: Card	after steps 1 to 3	by 60%	by 45%	by 45%
Gate 3: Proof	after steps 1 to 6	by 95%	by 95%	by 95%

If a gate has not passed at its checkpoint, you are not behind on *tools*. You are behind on *knowledge*. Go to the stuck clock.

The stuck clock

> **RULE** **If any step has used 1.5 times its share, stop.** Do not keep going harder. Run the four-question reset, and then ask someone, within 15 minutes.

The four-question reset:

1. **Is the expected behavior written down, with a source?** (If not, nothing else matters. Get it clarified.)
2. **Is the data on my card the data that matters?** Check by changing the data by hand and seeing whether the symptom responds.
3. **Can I make the problem smaller?** Fewer events, one instance, a shorter run (Agans: divide and conquer).
4. **What would I ask, and what is my best hypothesis?** Write both. (Alaboudi and LaToza: being offered a hypothesis made developers six times more likely to succeed in their experiment, so *bring one*.)

This is Agans's rule "Get a Fresh View" with a time limit attached. Asking after 15 minutes of structure is different from asking after four hours of flailing: you can hand the other person a card.

Adjusting for unknowns

From chapter 16 (also a heuristic):

- 0 to 2 unknowns: use the defaults.
- 3 to 5: add about a quarter to step 2.

- 6 or more: do not start probing. Split the task or get the expected behavior clarified.

Calibrate

Your journal (chapter 24) has columns for planned and actual minutes per step. After five tasks, look for patterns:

- A step that is *always* over: raise its share and lower another.
- A step that is *always* under: you are probably skipping part of it.
- Gate 1 failing late: your understanding step is shorter than it should be, or your sources (chapter 5) are too slow.

The goal is a split that is **yours**: one you can use to say "this will take about four hours, and here is where they go."

Check yourself

- What is the 4-hour split for a fix? For a feature?
- What triggers the stuck clock?
- What is the difference between being behind on tools and behind on knowledge?

PART 6

Proof by example

One small program. Three tasks: a fix, an understanding task, and a feature. Every command in these chapters was run, or is marked as not run.

CHAPTER 21

Worked example 1 · Fix

The task as handed over: “The link is reported Down for a few ticks even though heartbeats are arriving. Look at it around tick 27 in the test scenario.” Seven steps, one bug.

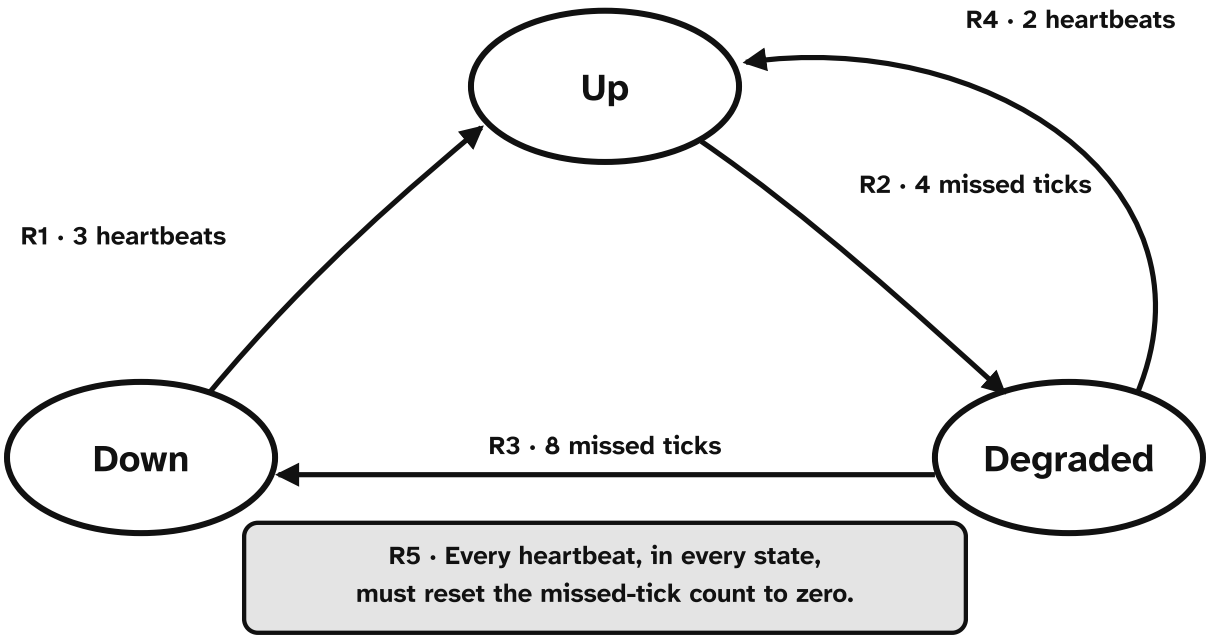


Figure 21.1. The link state machine as the requirements describe it. Chapter 21 finds the one place where the code does not honor R5.

Intake (5 minutes)

- **Family:** B (fix). **Type:** B3, “happened but shouldn’t”: a transition to Down that the rules forbid.
- **First probe:** FILTER at the end of the function that updates the data.
- **Time plan (4 h):** run 24 min, understand 48, state 36, probe 60, pin 24, change and prove 36, log 12. (The probes below each run in well under a second. The point is that the *plan* says where the time goes, so you notice when you spend it elsewhere.)
- **Unknowns on the card:** three (what the observed `missed_` was; whether every branch clears it; where exactly the Down comes from). That is within the “go” range.

Step 1 · Run it

```
cd lab && make && ./gateway --max 400
```

```
[t=  3] Down     -> Up       (missed=0 good=3)
[t= 23] Up       -> Degraded (missed=4 good=0)
[t= 27] Degraded -> Down     (missed=8 good=0)
[t= 29] Down     -> Up       (missed=0 good=3)
[t=123] Up       -> Degraded (missed=4 good=0)
[t=127] Degraded -> Down     (missed=8 good=0)
[t=134] Down     -> Up       (missed=0 good=3)
final: state=Up forwarded=54 dropped=6 data_seen=153
```

First oddity: `[t= 27] Degraded -> Down`, only four ticks after `Degraded`. The scenario has a gap in heartbeats for ticks 20 to 25, so Degraded at t=23 is right. Then heartbeats come back at t=26. The link should recover, not drop. The second outage (t=120 to 131) is a real outage and *does* go Down. So the output alone cannot tell the two apart. The requirements can.

Step 2 · Understand it

Rules R1 to R6 are in the comment block at the top of `gateway.cpp`. **R5** is the one at issue: *a link that is receiving heartbeats must not be declared Down*. The state-by-event table from chapter 5, built from the requirements, says that in **every** state a Heartbeat sets `missed` to 0.

Gate 1. *Predict*: "At the next Heartbeat while Degraded, `missed_` will be 0 and the link will go Up after two of them." That prediction is the claim the probes will test. Explain and locate pass (the card, chapter 6, names the module and the data).

Step 3 · State it

The card is in chapter 6. Its key lines:

```
I  (always)   after any Heartbeat is handled, missed_ == 0
P1 FILTER     stop after on_event if e.type == Heartbeat && missed_ != 0     predict:
never
```

Step 4 · Probe it RAN

```
breakpoint set -f gateway.cpp -l 81 -c 'e.type == EventType::Heartbeat && missed_ !=
0'
run --max 400
```

The prediction was wrong. The probe fired:

```
frame #0: gateway`LinkMonitor::on_event(this=…, e=…) at gateway.cpp:81:3
-> 81      }
(lldb) frame variable
(LinkMonitor *) this = 0x000000016fdfefec
(const Event &) e = 0x000000016fdfef90 (type = Heartbeat, tick = 26, payload = 0)
(lldb) p *this
(LinkMonitor)  (state_ = Degraded, missed_ = 7, good_ = 1, now_ = 26, data_seen_ = 8)
```

It stopped at **tick 26**, one tick *before* the visible Down. A Heartbeat was just handled, `state_` is `Degraded`, and `missed_` is still 7. That is the first bad state: the **infection**. The Down at t=27 is only the **failure**.

The equivalent gdb script is `lab/gdb/01-invariant.gdb`. **MANUAL**

Step 5 · Pin it

Follow the five moves from chapter 15.

- **Bad on entry or made inside?** `missed_` was 7 *before* the heartbeat (the previous tick made it 7), and it is still 7 after. The heartbeat handler did not change it. The data is not "bad on entry": 7 is a legitimate count after seven ticks without a heartbeat. The *handler's job* is to clear it. So the defect is inside `on_heartbeat`.
- **Which branch?** State is `Degraded`. `on_heartbeat` has three branches (Up, Down, Degraded).
- **Read the Degraded branch with the card beside it.** It increments `good_` and only clears `missed_` *inside* the `if (++good_ >= kRecoverAfter)` block. With one heartbeat, `good_` is 1, so `missed_` is not touched.

```
} else {  // Degraded
  if (++good_ >= kRecoverAfter) {
    LinkState from = state_;
    state_ = LinkState::Up;
    missed_ = 0;            // <-- only cleared here, after TWO heartbeats
    announce(from);
  }
}
```

The sentence: *`missed_` stays at 7 after the Heartbeat at t=26 because the Degraded branch of `on_heartbeat` clears it only after two consecutive heartbeats; the next Tick makes it 8 and the link goes Down.*

Experiment before editing. **RAN**

```
(lldb) expression missed_ = 0
(lldb) continue
[t= 27] Degraded -> Up       (missed=0 good=2)
```

With one value changed by hand, `Degraded -> Down` became `Degraded -> Up`, which is what R4 and R5 ask for. The sentence survived. Now you may edit.

Step 6 · Prove it RAN

The smallest change: clear `missed_` on every heartbeat in the Degraded branch. (Moving the `missed_ = 0` to the top of `on_heartbeat` for all states would also satisfy the rule. The fix below changes the least.)

```
     } else {  // Degraded
+      missed_ = 0;  // R5: a heartbeat always clears the missed-tick count
       if (++good_ >= kRecoverAfter) {
         LinkState from = state_;
         state_ = LinkState::Up;
-        missed_ = 0;
         announce(from);
       }
     }
```

Apply it (`patch -p1 < solutions/01-fix-missed-reset.patch`), rebuild, and run the baseline from step 1:

```
[t=  3] Down     -> Up       (missed=0 good=3)
[t= 23] Up       -> Degraded (missed=4 good=0)
[t= 27] Degraded -> Up       (missed=0 good=2)
[t=123] Up       -> Degraded (missed=4 good=0)
[t=127] Degraded -> Down     (missed=8 good=0)
[t=134] Down     -> Up       (missed=0 good=3)
final: state=Up forwarded=55 dropped=5 data_seen=153
```

The false Down is gone. The real outage (t=127) is unchanged. Now the **same probe**, on the fixed build: it was evaluated on **493 events** (`hit count = 493`) and **never fired**.

> **GATE** **Gate 3: Proof.** Two conditions, both met. (1) The probe *fired* on the broken build (at t=26), so we know it can detect the problem. (2) It stays silent on the fixed build across all 493 events. A probe that never fires proves nothing unless you have seen it fire.

Step 7 · Log it

```
JOURNAL 2026-10-16   gateway false Down        B3
Cause:   Degraded branch of on_heartbeat cleared missed_ only after 2 heartbeats (R5).
Fix:     missed_ = 0 at top of Degraded branch.  gateway.cpp:118
Proof:   probe 01 fired at t=26 before; silent for 493 events after. Output diff: t=27
Down -> Up.
Saved:   lab/lldb/01-invariant.lldb, lab/gdb/01-invariant.gdb
Learned: the visible failure (t=27) was one tick AFTER the first bad state (t=26).
Wrong hypothesis on the way: none this time (H1 held). Time: see plan vs actual in the
journal.
```

What made it fast

You never stepped through the loop. You never added a print. You never read a function you did not need. The card turned “the link drops when it should not” into one expression, and the debugger stopped on the first event that violated it.

Check yourself

- Why was the stop at t=26 more useful than the visible Down at t=27?
- What two things did the proof need?

CHAPTER 22

Worked example 2 · Understand

The task as handed over: “You are taking over `LinkMonitor`. Where is it called from? Is that the only place? Under what conditions? And how does it work?” This is the start of the book’s own wording: “I don’t know how this code works” and “I don’t know where this code is called.”

Intake

- **Family A.** Types **A1** (who calls this) and **A2** (how does it work). The deliverable is a map, not a change.
- **First probes:** LOG the callers, and tally which *(state, event)* combinations really occur.
- **Time plan (4 h, Family A):** run 24 min, understand 72, state 48, probe 60, pin 12, prove 12, log 12. Note how much more of the time goes to understanding than to probing.

Steps 1 and 2 · Run it, understand it

The run is the same as in chapter 21. The purpose, premise, and desired state are the five answers from chapter 5. The state-by-event table built from the requirements is the *expected* map. Our job is to compare it with what the program does.

Step 3 · State it (A1)

```
Q:       Who calls LinkMonitor::on_event? Only one place? Under what conditions?
DECLARE  LinkMonitor::on_event(const Event&)   gateway.cpp:74
DEFINE   caller set = {...}   each caller with the condition that leads to it
?        callers not exercised by a normal run
```

Step 4 · Probe it

First, the static answer

A text search shows what *exists* in the code. RAN

```
$ grep -n "on_event(" gateway.cpp
74:  void on_event(const Event& e) {
159:    monitor_.on_event(e);
170:      monitor_.on_event(extra);
```

One definition and **two call sites**: line 159 and line 170. Static search cannot tell how often each runs, or when.

Then the runtime answer

Log the caller of every call, and tally. In gdb this is the commands list from chapter 8. In lldb it is a Python callback. **RAN**

```
callers of LinkMonitor::on_event (function:line  count)
  Gateway::dispatch(Event const&):159     400
  Gateway::flush_burst(Event const&):170  93
```

Two callers, as the search said. `dispatch` is called once per event in the loop (400 events). `flush_burst` is the **rare path**: 93 calls, in groups of 3, so 31 bursts.

What selects the rare caller?

Break only when the caller is `flush_burst`. **RAN**

```
(lldb) bt
* frame #0: gateway`LinkMonitor::on_event(this=…, e=…) at gateway.cpp:75:12
  frame #1: gateway`Gateway::flush_burst(this=…, e=…) at gateway.cpp:170:16
  frame #2: gateway`Gateway::dispatch(this=…, e=…) at gateway.cpp:163:28
  frame #3: gateway`Gateway::run(this=…) at gateway.cpp:147:7
  frame #4: gateway`main(…) at gateway.cpp:186:6
(lldb) p e
(const Event &) 0x000000016fdfef60 (type = Data, tick = 90, payload = 99)
```

The first time it happens: tick 90, a `Data` event with `payload = 99`. Reading `dispatch` confirms the rule: a Data event with `payload >= 99` calls `flush_burst`, which feeds three extra Data events to `on_event`. The condition that selects the rare caller is **`Data`** **and** **`payload >= 99`**. In gdb: `break LinkMonitor::on_event if $_caller_matches(".*flush_burst")`, or, without Python, `break gateway.cpp:170`. **MANUAL**

EVIDENCE **A finding that reading alone would likely have missed.** The summary line says `data_seen=153`, but `forwarded=54` plus `dropped=6` is 60. Two counters that look like they should agree do not. The caller tally explains it: 153 − 60 = **93**, exactly the number of calls from `flush_burst`. `data_seen_` counts the synthetic burst events as well. Whether that is a bug or intended is a question for the owner. The point is that the probe found the question.

Step 4 (continued) · A2: how does it work?

Tally every *(state, event)* pair that occurs. **RAN** Compare with the table from the requirements.

```
observed state x event (count of events seen in that state)
                 Tick  Heartbeat     Data
  Down             12          9        9
  Up              160        150      135
  Degraded          8          1        9
```

What this tells you, and what it does not:

- **Every cell was reached at least once.** The scenario exercises the whole table.
- **`Degraded` by `Heartbeat` was reached exactly once.** It is the thinnest cell, and the one with the bug (chapter 21). A cell with one observation is a claim you have barely tested.
- **Counts show coverage, not correctness.** To check correctness you compare each cell's *outcome* with the requirement table. For `Degraded` by `Heartbeat` the requirement says `missed = 0`, and the FILTER probe in chapter 21 showed it was not.

Step 5 · Pin it

For an understanding task, "pinned" means the map is complete: who calls, under which conditions, and where the state changes. The writer log from chapter 11 gives the last piece: state changes happen at **four sites** in the code, `on_tick` (2) and `on_heartbeat` (2), and only three of them ran in this scenario.

Step 6 · Prove it (explain it back)

For Family A, "prove" means you can *predict*. Before running the next scenario, write down: "The next time `Data` arrives with `payload >= 99`, `data_seen_` will increase by 4 (1 plus 3 burst events) while `forwarded_` or `dropped_` increases by 1." Then check it.

Step 7 · The deliverable

```
MODULE      LinkMonitor  (gateway.cpp)
PURPOSE     Decide link health from ticks and heartbeats so data flows only on a
healthy link.
STATE       state_ {Down, Up, Degraded}, missed_, good_
EVENTS      Tick, Heartbeat, Data                    (state x event table: see chapter
5)
CALLED BY   Gateway::dispatch   every event          (gateway.cpp:159)
            Gateway::flush_burst rare: Data with payload >= 99, 3 synthetic Data
events (gateway.cpp:170)
WRITERS     state_ at lines 96, 101, 114, 120        (on_tick 2, on_heartbeat 2)
OPEN        data_seen_ includes burst events: intended?   Only 1 observation of
Degraded x Heartbeat.
```

RULE **Static and runtime together.** Search showed what exists. The probe showed what ran. The gap between them (in this case, a rare path and a rarely-seen cell) is where the surprises are.

Check yourself

- Which two tools gave you the callers, and what does each one not tell you?
- How did the caller tally explain the mismatch between `data_seen` and `forwarded + dropped`?

CHAPTER 23

Worked example 3 · Build

The task as handed over: “When the link recovers to Up, log how long the outage lasted.” (This runs on the fixed build from chapter 21.) The skill is the same: describe the new behavior as before and after states, then find every place the change belongs.

Intake

- **Family C.** Types **C1** (new behavior on a condition: when the link becomes Up) and **C2** (new data: when the outage began).
- **First probe:** LOG every writer of `state_` with its line and call chain.
- **Time plan (4 h, Family C):** run 12 min, understand 60, state 36, probe 36, pin 24, build and prove 60, log 12.

Step 1 · Run it

Run the fixed build and look at the nearest existing behavior: the `X -> Up` lines.

```
[t=  3] Down     -> Up       (missed=0 good=3)
[t= 23] Up       -> Degraded (missed=4 good=0)
[t= 27] Degraded -> Up       (missed=0 good=2)
[t=123] Up       -> Degraded (missed=4 good=0)
[t=127] Degraded -> Down     (missed=8 good=0)
[t=134] Down     -> Up       (missed=0 good=3)
```

Two recoveries: **t=27** (from Degraded) and **t=134** (from Down, after a real outage). The feature must report on both. The first line, t=3, is the initial connection, not a recovery. It should *not* report an outage.

Steps 2 and 3 · Understand and state

```
BEHAVIOR CARD — outage duration                          Family C · C1 + C2
RUN       ./gateway --max 400  -> "[t=134] Down -> Up"  (no duration printed today)
EXPECTED  When the link returns to Up after being off Up, print how many ticks it was
off Up.
PURPOSE   Let an operator see how long each outage lasted.

DECLARE
  D1  state_         LinkState    exists (gateway.cpp:132)
  D2  now_           int          exists: current tick
  D3  outage_start_  int          NEW. tick when we left Up. -1 means "no outage
open".

DEFINE
  P   state_ changes from Up to Degraded               -> set outage_start_ = now_
  Q   state_ changes to Up from Down OR from Degraded  -> print now_ - outage_start_,
set -1
  I   outage_start_ >= 0 exactly while state_ != Up (except before the first connect)
  Q'  today: nothing printed

IMPLEMENT
  ? Every place state_ is written. Which of them are "leave Up" and which are "enter
Up"?

PROBES
  P1 LOG+TRAP   log every write to state_ with line and caller      predict: I see the
writers
```

Step 4 · Probe it RAN

The writer log from chapter 11, on the **fixed** build. The call chain is shortened here (`via` shows the function that wrote). The values and line numbers are the real output. The annotations in parentheses are mine, read from the code at those lines.

```
state_ written -> Up        after line 115   via on_heartbeat   (Down -> Up)
state_ written -> Degraded  after line 97    via on_tick        (Up -> Degraded)
state_ written -> Up        after line 122   via on_heartbeat   (Degraded -> Up)   <-
a new site: it ran this time
state_ written -> Degraded  after line 97    via on_tick
state_ written -> Down      after line 102   via on_tick        (Degraded -> Down)
state_ written -> Up        after line 115   via on_heartbeat   (Down -> Up)
```

A watchpoint reports the line *after* the write, so the writes are at lines 114, 96, **121**, 96, 101, 114 in the fixed file. Compare with chapter 11, where the same log, on the **buggy** build, had no line-122 entry. The fix made a fourth write site reachable.

Step 5 · Pin it: the change-point table

Where (fixed file, file:line)	What happens there	Seen in the run?	Needs the change?	Why
`on_tick` 96	Up → Degraded	yes	**yes**	the outage starts: set `outage_start_ = now_`
`on_tick` 101	Degraded → Down	yes	no	still the same outage
`on_heartbeat` 114	Down → Up	yes	**yes**	the outage ends: report and clear
`on_heartbeat` 121	Degraded → Up	**no, before the fix of chapter 21**	**yes**	the outage ends here too: report and clear

TRAP **The path the bug hid.** With the original (buggy) code, the Degraded → Up write (line 120 in the original file, 121 after the fix) never ran in this scenario. A feature hooked only where the log showed "Up" would have covered one of the two ways to recover. It would have *looked* complete in the test run, and been wrong in production the first time a short outage ended. This is why the table has a column for "seen in the run." Rows marked *no* are decided by reading the code.

Step 6 · Build and prove RAN

The change has three parts, one for each row marked *yes*, plus the new field and a helper that reports and clears it (full patch: `solutions/02-feature-outage-duration.patch`).

```
       state_ = LinkState::Degraded;
       good_ = 0;
+      outage_start_ = now_;                 // outage begins
       announce(from);
 ...
         state_ = LinkState::Up;             // Down -> Up
         announce(from);
+        report_outage();
 ...
         state_ = LinkState::Up;             // Degraded -> Up
         announce(from);
+        report_outage();
 ...
+  void report_outage() {
+    if (outage_start_ < 0) return;          // first connect: there was no outage
+    std::printf("[t=%3d] link recovered after %d ticks\n", now_, now_ -
outage_start_);
+    outage_start_ = -1;
+  }
+  int outage_start_ = -1;
```

New behavior, in the run:

```
[t=  3] Down     -> Up       (missed=0 good=3)
[t= 23] Up       -> Degraded (missed=4 good=0)
[t= 27] Degraded -> Up       (missed=0 good=2)
[t= 27] link recovered after 4 ticks
[t=123] Up       -> Degraded (missed=4 good=0)
[t=127] Degraded -> Down     (missed=8 good=0)
[t=134] Down     -> Up       (missed=0 good=3)
[t=134] link recovered after 11 ticks
```

- **t=27:** 27 − 23 = 4 ticks. Correct, and from the *Degraded* path.
- **t=134:** 134 − 123 = 11 ticks. Correct, and from the *Down* path. The Down in the middle did not restart the clock.
- **t=3:** no report. The sentinel `-1` handled the first connect.

Old behavior unchanged. Remove the new lines from the output and compare with the output before the feature:

```
$ diff <(./gateway_feature --max 400 | grep -v recovered) <(./gateway_fixed --max 400)
$                                  # empty: identical apart from the new lines
```

RAN Both ways into `Up` are covered, and nothing that already worked changed.

GATE **Gate 3: Proof for a feature** means two things: (1) the new behavior shows up on *every* path in the change-point table that your runs reach, and (2) the old behavior is byte-for-byte unchanged. Run a scenario that reaches each *no* row, or reason about it and say so in the journal.

Step 7 · Log it

```
JOURNAL   outage duration                C1 + C2
Change points: gateway.cpp 96 (start), 114 and 121 (report). Table in the card.
Hidden path:   Degraded -> Up (line 121 in the fixed file) was unreachable until the
R5 fix. Feature depends on it.
Proof:         t=27 -> 4 ticks, t=134 -> 11 ticks, t=3 silent. Output diff vs fixed
build: empty.
Saved:         lab/lldb/02-log-state-writes.lldb, solutions/02-feature-outage-
duration.patch
Open question: should "Down" lasting longer than N ticks raise an alarm? (new card,
not this one)
```

Check yourself

- Why does the change-point table have a “seen in the run” column?
- What would have gone wrong if the feature had been built before the fix in chapter 21?

PART 7

Habits

Steps 6 and 7 in full, then how to work with other people and with AI tools without losing the discipline.

CHAPTER 24

Steps 6 and 7 · Prove it and log it

The work is not finished when the symptom disappears. It is finished when you can show why, and when the next person can repeat it. Proof makes the result trustworthy. The log makes the next task faster.

Step 6 · Prove it

The seven checks

1. **The sentence was tested first.** You changed a value by hand and saw the symptom respond (chapter 15). If not, go back.
2. **The change is the smallest one that makes the sentence false.** One change at a time (Agans: “If you didn’t fix it, it ain’t fixed”, and if you changed two things you cannot say which one did).
3. **The baseline run behaves.** Re-run step 1 exactly. `diff` the output against the saved run. The *only* differences should be the ones you intended.
4. **The same probes are silent.** Re-run the probes from your card. Read their **hit counts**. In chapter 21 the probe was evaluated 493 times and never stopped. That number matters: it proves the probe was alive.
5. **The probe could fire.** You saw it fire on the broken build. (Keep the old binary, or `git stash`, to confirm.) A silent probe means nothing until you have seen it fire.
6. **Neighbors are unchanged.** Run the tally probe (chapter 22) before and after, and diff. Cells you did not mean to touch must not move.
7. **The card is now code.** Next section.

Turn the card into code: the permanent probe

A probe in your debugger disappears. The same condition, written as an `assert`, stays forever and runs on every build. The DEFINE line of your card is already the expression.

```
// at the end of LinkMonitor::on_event:
// R5 as an executable invariant: after any Heartbeat is handled, missed_ is 0.
assert(!(e.type == EventType::Heartbeat && missed_ != 0) && "R5 violated");
```

I built the practice program both ways. **RAN**

```
(buggy build)
Assertion failed: (!(e.type == EventType::Heartbeat && missed_ != 0) && "R5
violated"),
  function on_event, file gateway.cpp, line 83.            exit status: 134

(fixed build)
final: state=Up forwarded=55 dropped=5 data_seen=153        exit status: 0
```

The bug now cannot return silently. This is the connection to **Design by Contract**: the invariant that was in your head is in the code, where it is checked, read, and run by everyone. For a longer-lived guard, or for code where an `assert` is compiled out in release builds, write a **unit test** with the same condition.

Proof for a fix versus a feature

	Fix	Feature
Show	The first bad state no longer occurs	The new behavior occurs on every path
And	The probe is silent and was seen to fire	The old behavior is unchanged (diff)
Keep	An assert or test of the invariant	A test for each path in the change-point table

GATE **Gate 3: the Proof Gate.** Checks 1 to 6 pass, and you have decided what stays behind (an assert, a test, a saved probe). If you cannot say what would catch this if it came back, you are not done.

Step 7 · Log it

You do this *after every task*, in about 5 percent of the time. It is the cheapest step and the one most often skipped. Agans: “write down what you did, what order you did it in, and what happened as a result.”

The journal entry

```
JOURNAL  <date>   <task>                    type: ___
Cause / change:   one sentence ("X is Y because line L ...")
Fix / feature:    file:line, commit
Proof:            what you ran, what you saw (counts, diff)
Saved:            probe scripts, test names
Learned:          what you would tell the next person
First wrong assumption:  _____________________________
```

The Hypothesis Ledger

Alaboudi and LaToza found that people hold few hypotheses (about two per bug) and that a correct early hypothesis predicts success. Writing them down makes the first one explicit and the wrong ones cheap.

#	Hypothesis	Prediction	Probe	Result
H1	Degraded branch does not clear `missed_`	`missed_` is 7 after the t=26 heartbeat	P1 FILTER	**confirmed**
H2				

Plan versus actual

Step	Planned (min)	Actual (min)	Note
1 Run			
2 Understand			
3 State			
4 Probe			
5 Pin			
6 Prove			
7 Log			

After five tasks, read the “Actual” column across all of them. Which step is always over? Which is always skipped? That is how the budgets in chapter 20 become yours.

Weekly, five minutes

Read the “First wrong assumption” line from each entry. Is there a pattern? A pattern here is worth more than any single fix.

Commit messages

Put the sentence in the message. Whoever runs `git blame` later will thank you.

```
Fix false Down: clear missed_ on every heartbeat (R5)

Degraded branch cleared missed_ only after 2 heartbeats, so the next Tick
pushed it to 8 and the link went Down while heartbeats were arriving.
Found with a conditional probe at the end of on_event
(tools/probes/stale-missed.gdb); probe is silent on 493 events after the fix.
```

Check yourself

- Which of the seven checks do people skip most? Which would you skip under pressure?
- What does a hit count prove that a silent run does not?
- What stays in the code after you finish?

CHAPTER 25

Working with other people, and with AI

The method is personal, but the output is shareable. A card is something you can hand to a colleague. A probe file is something a teammate can run. This chapter is about using that, and about using AI tools without giving up the part only you can do.

Share the card, not the confusion

- **In a ticket or pull request:** paste the **sentence**, the **card** (or just EXPECTED, OBSERVED, and the probe), and the **proof**. Reviewers can check the reasoning, not only the diff.
- **When you ask for help:** hand over the card with the `?` marks. “Here is what I expect, here is what I see, here is the probe I ran, here is my hypothesis, and here is where I am stuck” takes a minute to read and gets better answers than “it’s flaky.” (In Alaboudi and LaToza’s experiment, being offered a plausible hypothesis was the aid that helped; a bare location did not.)
- **In handoffs:** the Understand deliverable (chapter 17) is the handoff document.

Ask for a fresh view on a schedule

Agans lists “Get a Fresh View” as a rule because the person who has been staring at a problem has a stale picture of it. Build it into the clock (chapter 20): when a step reaches 1.5 times its budget, you ask, within 15 minutes. Say so up front: “my check-in is at 2 hours.”

Put probes where the team can find them

- `tools/probes/` in the repository, or a shared notes folder, whichever your team uses.
- One file per probe, with the card reference and the prediction in the header (chapter 14).
- A short `README` listing the file names and what each answers. A probe nobody can find is not shared.

Code review with cards

When you review someone else's fix, ask three questions from the method:

1. What is the **first bad state**, and where was it observed?
2. What **probe** showed it, and was it seen to fire *before* and stay silent *after*?
3. Which **other paths** reach this code, and how do we know?

Contracts in the code

Where a card's DEFINE line is a rule the system must always obey, make it **executable**: an `assert`, a unit test, or a runtime check that logs loudly (chapter 24). Bertrand Meyer's Design by Contract gives the vocabulary: preconditions, postconditions, class invariants. You have been writing them on cards. Moving the important ones into the code is how a team stops re-discovering the same bug.

Using AI tools

AI assistants can help with this method. They can also tempt you to skip it. Here is where they fit.

Good uses

- **Draft a card** from a code excerpt and a log. You check every line against the run.
- **Explain an unfamiliar API or idiom** you hit while reading (a template, a macro, a library call).
- **Draft a probe script** (a gdb command file, a Python breakpoint class) from the card. Then **run it, read gdb's errors, and verify**. Commands from any source, including this book, should pass your own smoke test (Appendix C).
- **Suggest hypotheses.** Treat them as input to the ledger, not as answers.
- **Rewrite your journal** into a clean summary or a commit message.

What not to delegate

- **Step 1.** Running the program and looking at the output is yours. Without it you will accept a plausible explanation of behavior you never saw.
- **The Confidence Gate.** If you cannot explain it, predict it, and locate it, an AI's explanation does not change that. It gives you something to *test*.
- **The expected behavior.** Only the spec, the ticket, or a person knows what *should* happen.

- **The proof.** The probe must fire on the old build and stay silent on the new one, on your machine.

> **RULE** **An AI can propose a hypothesis. Only a probe can confirm it.** Put the proposal on the ledger, write the prediction, and run the probe.

> **TRAP** **Work code and external tools.** Pasting proprietary code, logs, or data into an external AI service can break your employer's rules. I do not know your employer's policy. Ask before you paste anything from a work code base, and use only the tools your team has approved. The method does not require any AI tool.

Your `handbook/` chapter 14 covers AI assistance in more depth.

Check yourself

- What do you hand someone when you ask for help?
- What are the three questions you ask in review?
- Which two steps do you never hand to an AI tool?

APPENDICES

Reference

VS Code setup, the gdb and lldb command map, a smoke test for your Linux machine, the evidence behind the book, and a glossary.

APPENDIX A

VS Code over SSH: the step-by-step

Your setup: VS Code on a Windows laptop, connected by SSH to a Linux server. You installed the C/C++ extension on the server, wrote a launch.json, and when you pressed debug it ran from the terminal with no connected UI. This appendix is a working setup and a ranked list of things to check.

> **NOTE** **What I could and could not do.** I cannot see your machines. The steps below come from Microsoft's VS Code documentation (Remote-SSH, C/C++ launch.json reference, Linux configuration page). I did not run them against a Windows-to-Linux setup. The troubleshooting table lists *hypotheses to test*, not a diagnosis. The gdb in the terminal always works; the GUI is a convenience.

Where each piece runs

Piece	Runs on	Notes
VS Code window (the UI)	Windows laptop	What you see and click
VS Code Server, extensions, terminal	Linux server	Started for you by Remote-SSH
C/C++ extension (`cppdbg`)	**Linux server**	Must be installed *on the remote*
`gdb`, `g++`, your code, the program	Linux server	`gdb` and the compiler must exist here

The Microsoft docs say: "Local extensions that actually need to run remotely will appear dimmed and disabled in the Local - Installed category. Select Install to install an extension on your remote host."

Setup, in order

1. Connect. Install the *Remote - SSH* extension locally. In VS Code, run **Remote-SSH: Connect to Host...**, choose your server. A new window opens with the host name in the bottom-left corner.

2. Open your folder on the remote. File → Open Folder and pick the project directory *on the server*. The debug configuration lives in `.vscode/` inside it.

3. Install the C/C++ extension on the remote. In the Extensions view of the *remote window*, find **C/C++** (Microsoft) and select **Install** for your host. If the button says "Install in SSH: your-host", that is the one.

4. Check the server has the tools. In the remote integrated terminal:

```
gdb --version
g++ --version
which gdb            # note the path for miDebuggerPath
```

If `gdb` is missing: `sudo apt-get install build-essential gdb` (Debian/Ubuntu, from the VS Code Linux page).

5. Build with symbols. `-g -O0` (chapter 4). For the lab: `make -C lab CXX=g++`. Confirm symbols exist:

```
gdb -batch -ex "list main" lab/gateway      # should print source lines
```

6. Create `.vscode/launch.json`.

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Debug gateway (gdb)",
      "type": "cppdbg",
      "request": "launch",
      "program": "${workspaceFolder}/lab/gateway",
      "args": ["--max", "400"],
      "stopAtEntry": false,
      "cwd": "${workspaceFolder}/lab",
      "externalConsole": false,
      "MIMode": "gdb",
      "miDebuggerPath": "/usr/bin/gdb",
      "setupCommands": [
        { "description": "Pretty-print STL", "text": "-enable-pretty-printing",
"ignoreFailures": true }
      ],
      "preLaunchTask": "build gateway"
    }
  ]
}
```

7. Create `.vscode/tasks.json` (the build step that runs first).

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "type": "shell",
      "label": "build gateway",
      "command": "make",
      "args": ["-C", "${workspaceFolder}/lab", "CXX=g++"],
      "problemMatcher": ["$gcc"],
      "group": { "kind": "build", "isDefault": true }
    }
  ]
}
```

8. Set a breakpoint and start the right way. Click in the gutter next to a line in `gateway.cpp`. Open **Run and Debug** (the bug-and-play icon), choose **Debug gateway (gdb)** in the dropdown, and press the green **Start Debugging** button (F5).

> **TRAP** **Do not use "Run Without Debugging" (Ctrl+F5), a "Run" button from another extension, or the terminal.** Those start the program with no debugger attached. They look similar and are the most common reason the UI never connects.

9. Check that the UI is connected. You should see: a yellow arrow on the stopped line, the **Variables** pane (expand `this` to see members), a **Call Stack**, and a **Debug Console**. In the Debug Console, `-exec info breakpoints` should list your breakpoint.

10. Try the features from this book in the UI. Right-click the gutter: **Add Conditional Breakpoint** (an expression, a hit count, or "wait for breakpoint"), or **Add Logpoint** (text with `{expressions}`). Per the VS Code docs, data breakpoints "depend on the debugger", so test whether your setup offers them. If not, use `-exec watch -l …` in the Debug Console.

If it ran in the terminal with no UI: hypotheses, most likely first

These are **things to test**, not conclusions. Work down the list.

#	Hypothesis	How to test	If true
1	The program was started some other way (terminal, `make run`, Ctrl+F5, another extension's Run button)	Open the Run and Debug panel. Is **Debug gateway (gdb)** in the dropdown? Did you press F5 with that selected?	Use the green Start Debugging with that configuration

#	Hypothesis	How to test	If true
2	`type` or `request` is wrong in `launch.json`	The dropdown shows no configuration, or a different debugger. `type` must be `cppdbg`, `request` must be `launch`	Fix the JSON
3	The C/C++ extension is not installed on the **remote**	Extensions view in the remote window: does it say **Install** (not "Installed") next to C/C++? Does the dropdown lack `C++ (GDB/LLDB)`?	Install it on the SSH host
4	`externalConsole` is true, so output goes to another window	Check the property	Set `"externalConsole": false`
5	Breakpoints are hollow or grey ("unbound")	Hover the breakpoint. Was the binary built with `-g -O0`? Is `program` the same file you built?	Rebuild with symbols; fix `program`
6	`miDebuggerPath` is wrong or gdb is missing	A pop-up or error in the **Debug Console** or **Output → C/C++**	Use the path from `which gdb`
7	The adapter failed silently	Add the `logging` options from the launch.json reference and read **Output → C/C++**	Follow the message
8	Attach errors: `ptrace: Operation not permitted`	Only when *attaching* to a running process	See the VS Code C++ debugging page; may need ptrace permission
9	Corporate restrictions on installing extensions on the remote	Install fails or is blocked	Ask your team about an approved offline install route. *Plausible, unverified.*

The terminal is always an option

Everything in this book works in a plain terminal:

```
gdb -q ./lab/gateway
(gdb) break gateway.cpp:81 if e.type == EventType::Heartbeat && missed_ != 0
(gdb) run --max 400
```

If the GUI costs you more time than it saves on a given day, use the terminal and come back to it.

Sources

VS Code docs: *Debugging* (breakpoint types), *C/C++ launch.json reference*, *Configure VS Code for debugging on Linux*, *Remote - SSH*, and *C/C++ debugging* (the `-exec` note). See Appendix D.

APPENDIX B

Command map: gdb and lldb

The commands used in this book, side by side. A mark means I ran it (RAN, lldb on macOS) or checked it against the GDB manual (MANUAL). Unmarked lldb entries are from general knowledge and were not run.

Run, stop, move

Goal	gdb	lldb
Start with arguments	`run --max 400` MANUAL	`run --max 400` RAN
Stop at a function	`break Gateway::run` MANUAL	`breakpoint set -n Gateway::run` RAN
Stop at file and line	`break gateway.cpp:81` MANUAL	`breakpoint set -f gateway.cpp -l 81` RAN
One-shot	`tbreak LOC` MANUAL	`breakpoint set … -o true` RAN
Continue	`continue`	`continue` RAN
Finish this function (show return)	`finish`	`finish`
Stack	`bt`, `bt 5`, `bt full`	`bt`, `bt 5` RAN
Look at the caller / back	`up`, `down`, `frame N`	`up` RAN, `down`, `frame select N`
Kill the program	`kill`	`kill` RAN

Conditions, counts, logs

Goal	gdb	lldb
Conditional breakpoint	`break LOC if COND` MANUAL	`breakpoint set … -c 'COND'` RAN
Add or change a condition	`condition N COND` MANUAL	`breakpoint modify -c 'COND' N`
Ignore the next C hits	`ignore N C` MANUAL	`breakpoint set … -i C` RAN
Hit counts	`info breakpoints` MANUAL	`breakpoint list` RAN

Goal	gdb	lldb
Print and keep going	`dprintf LOC,"fmt",args` MANUAL	callback returning False RAN
Run commands at a stop	`commands N` … `end` MANUAL	`breakpoint command add -F mod.fn N` RAN
Counter	`set $n = $n + 1` MANUAL	Python variable in a callback RAN
By caller	`$_caller_matches(".*fn")` MANUAL	callback checks parent frame RAN
Disable, enable, delete	`disable N`, `enable N`, `delete N`	`breakpoint disable N`, …
Enable once / count	`enable once N`, `enable count C N` MANUAL	
Save and reload	`save breakpoints F`, `source F` MANUAL	

Watchpoints

Goal	gdb	lldb
Watch for writes	`watch EXPR` MANUAL	`watchpoint set variable EXPR` RAN
Watch the memory location	`watch -l EXPR` MANUAL	`watchpoint set variable this->a.b` RAN
Watch reads / reads and writes	`rwatch`, `awatch` MANUAL	`watchpoint set variable -w read_write …`
Conditional watchpoint	`watch EXPR if COND` MANUAL	`watchpoint modify -c 'COND' N` RAN
Run commands on a hit	`commands N`	`watchpoint command add -F mod.fn N` RAN
List and remove	`info watchpoints`, `delete N`	`watchpoint list`, `watchpoint delete N` RAN

Looking at and changing state

Goal	gdb	lldb
Arguments	`info args` MANUAL	`frame variable` RAN
Locals	`info locals` MANUAL	`frame variable` RAN

Goal	gdb	lldb
Whole object	`print *this`	`p *this` **RAN**
One value	`print missed_`	`p missed_` **RAN**
Type definition	`ptype T`	`type lookup T` **RAN**
Pretty output	`set print pretty on` **MANUAL**	(default)
Change a value by hand	`set var missed_ = 0` **MANUAL**	`expression missed_ = 0` **RAN**
Search functions	`info functions Link`	`image lookup -r -n Link` **RAN**

Time travel, crashes, scripts

Goal	gdb	lldb
Reverse	`reverse-continue`, `reverse-step`, `reverse-next`, `reverse-finish` (after `record`, or under rr) **MANUAL**	not used here
Catch a C++ exception / a signal	`catch throw`, `catch signal` **MANUAL**	`breakpoint set -E c++`
Core dump	`gdb ./prog core`	`lldb ./prog -c core`
Run a script, then exit	`gdb -q -batch -x f.gdb ./prog` **MANUAL**	`lldb -b -s f.lldb ./prog` **RAN**
Python probe	`class P(gdb.Breakpoint): def stop(self)` **MANUAL**	`breakpoint command add -F mod.fn` **RAN**
Load a Python file	`source file.py`	`command script import file.py` **RAN**

Convenience: breakpoint numbers and variables in gdb

`$bpnum` holds the number of the last breakpoint you created. `$_hit_bpnum` holds the one just hit (`disable $_hit_bpnum` turns off whichever breakpoint stopped you). `init-if-undefined $n = 0` sets a convenience variable only if it does not exist. **MANUAL**

APPENDIX C

Smoke test for your Linux machine

I could not run gdb on the machine where I wrote this book. Every gdb command here was checked against the official manual, not executed. This two-minute test closes that gap on *your* gdb.

Run it

```
cd lab
make CXX=g++
sh gdb/smoke-test.sh
```

It prints your gdb version, whether gdb has Python, and one line per probe:

```
GNU gdb (…) 12.1
gdb has Python: yes
PASS  01-invariant
PASS  02-log-state-writes
PASS  03-rare-caller
PASS  04-counterfactual
PASS  05-count
PASS  06-arm-then-watch
PASS  07-break-on-output
PASS  08-python-probe
PASS  09-callers
```

(That is the shape of the output, not a captured result.) Each probe's full output is saved in `lab/transcripts/gdb-NN-name.txt`.

If a probe fails

1. Open its transcript. Read gdb's message. It is usually specific.
2. Find the message in the table below.
3. Fix the script. Then tell me the message, and I will fix the book.

gdb says	It means	Do this
`No symbol table is loaded. Use the "file" command.`	The binary has no debug info	Rebuild with `-g -O0`
`Make breakpoint pending on future shared library load?`	The name or file was not found	Check spelling; check `info functions NAME`

gdb says	It means	Do this
`Python scripting is not supported in this copy of GDB.`	gdb built without Python	Probes 3 and 8 need a Python build. Use the fallback in probe 3 (`break gateway.cpp:170`)
`Function "$_caller_matches" not defined.`	Same: no Python	Same fallback
`Could not insert hardware watchpoint`	Too many hardware watchpoints for this target	Delete some; or `set can-use-hw-watchpoints 0` (slow)
`Cannot watch constant value`	You gave an address literal	`watch *(int *) 0xADDR`
`<optimized out>`	Built with optimization	Rebuild with `-O0`
`No frame selected.`	The program is not running	`run` first
A watchpoint keeps firing after the program ends	The watched stack memory is being reused	Delete it at a breakpoint near the end (the lab scripts do)
Line-number probes stop at the wrong place	The file changed	Use function names, or re-check the line with `list gateway.cpp:75,81`

Quick checks you can run in 10 seconds

```
gdb --version                              # which gdb
gdb -batch -ex 'python print(1)'           # does it have Python? prints 1 if yes
gdb -batch -ex 'list main' ./gateway       # are symbols present?
```

What the smoke test does not check

- It does not test your *own* project. It tests that your gdb understands the forms used here.
- It does not test VS Code. Appendix A covers that.
- It does not test rr, core dumps, or reverse execution (chapter 13).

What I verified, and how

Where	What I did
lldb on macOS	Ran every lldb probe in `lab/lldb/` and saved the output in `lab/transcripts/`.
gdb	Checked syntax against the GDB manual pages listed in `resource/research/03-gdb-mechanics.md`. Did not execute.
Patches	Applied in order with `patch -p1`, built, and ran.
Assert example	Built both ways and ran (chapter 24).

APPENDIX D

Evidence and sources

What each claim in the book rests on, how strong it is, and what could not be verified. The full notes, with quotes, are in `resource/research/`.

How to read the strength column

- **Primary**: I read the source's own text.
- **Secondary**: I read a reputable page that quotes or summarizes it. SEC
- **Manual**: tool behavior from the official manual, not executed. MANUAL
- **Ran**: executed here. RAN

Empirical studies

Claim in the book	Source	Strength	Caveat
Developers did not use the debugger in 91% of IDE sessions; advanced features under-used	Beller, Spruit, Spinellis, Zaidman, "On the Dichotomy of Debugging Behavior Among Programmers", ICSE 2018	Primary	Java, Eclipse and IntelliJ. 176 survey respondents, 458 instrumented developers
"once one knows the condition, one almost automatically understands the problem"	Same paper, expert interview	Primary	Two interviewees; a quote, not a measurement
Correct early hypothesis predicts success; offered hypotheses helped, offered locations did not	Alaboudi and LaToza, "Using Hypotheses as a Debugging Aid", VL/HCC 2020	Primary (abstract read; arXiv)	20 developers; lab tasks
About 58% of time on program comprehension	Xia et al., "Measuring Program Comprehension...", IEEE TSE 2018	Secondary (publisher abstract)	78 developers, 3,148 hours, 7 projects; an average
Methodical, structured investigation is most effective	Robillard, Coelho, Murphy, IEEE TSE 2004	Primary (abstract read)	5 developers, one system

Claim in the book	Source	Strength	Caveat
Why-did / why-didn't questions; Whyline cut debugging time "nearly a factor of 8"	Ko and Myers, "Designing the Whyline", CHI 2004	Primary	Alice teaching environment; six scenarios; an upper bound
Systematic readers did better on small programs; experts on large ones read as-needed	Littman et al. 1986; Koenemann and Robertson 1991, as described in a 2023 survey (arXiv 2310.11301)	Secondary	Original papers not read

Authorities and books

Claim in the book	Source	Strength
A programmer's theory: explain, justify, respond to change; decay without it	Naur, "Programming as Theory Building", 1985	Primary (mirror text)
Scientific method for debugging; defect → infection → failure	Zeller, *Why Programs Fail*, ch. 6	Primary (excerpt)
TRAFFIC: Track, Reproduce, Automate, Find origins, Focus, Isolate, Correct	Zeller	Secondary SEC
Nine rules; "quit thinking and look"; keep an audit trail	Agans, *Debugging* (2002)	Secondary (a review quoting the book) SEC
Four phases; unknown, data, condition	Pólya, *How to Solve It*	Secondary (Wikipedia; course page) SEC
"Data dominates"; measure first	Pike, "Notes on Programming in C", 1989	Primary
Show the tables / data structures first	Brooks (*Mythical Man-Month* ch. 9); Torvalds (git list, 2006, via LWN)	Secondary; Brooks paraphrased, wording varies by edition
Behaviors as sequences of states; specs above the code	Lamport (TLA+; "Thinking for Programmers")	Secondary (abstract and interview coverage) SEC
`{P} C {Q}`; class invariants; Design by Contract	Hoare; Meyer	Secondary (lecture notes) SEC
Characterization tests; seams	Feathers, *Working Effectively with Legacy Code*	Secondary SEC
Hypothetico-deductive troubleshooting; bisection; "what touched it last"	Google, *Site Reliability Engineering*, ch. 12	Primary

Claim in the book	Source	Strength
Follow the data; run it first; conditional watchpoints	Shah and Croft-White, C++ on Sea 2025	Primary (your transcript)

Tools

Claim in the book	Source	Strength
Break conditions, ignore counts, command lists, `dprintf`, watchpoint rules, convenience functions, Python breakpoints, reverse execution, startup and batch options	GDB manual (sourceware.org, current docs)	Manual MANUAL
Cortex-M hardware watchpoints; late halt; same-value writes suppressed	Memfault Interrupt, “Faster Debugging with Watchpoints”	Primary (first ~39k characters)
gdb-dashboard	github.com/cyrus-and/gdb-dashboard README	Primary
rr record, replay, reverse commands, limits	rr-project.org and its wiki	Primary
VS Code launch.json, Remote-SSH, breakpoint types, `-exec`	VS Code documentation	Primary
lldb behavior	lldb-1700.0.9.502, this Mac	Ran RAN

What I could not verify, and what I did about it

Item	What happened	Handling
Whether any gdb command *runs*	No gdb on the machine used	Manual-checked; smoke test (Appendix C)
`$_caller_matches` anchoring	The manual does not say	Pattern written to work either way
Local `./.gdbinit` auto-loading	Not confirmed for local files	The book teaches `-x`
VS Code data-breakpoint support for gdb	Docs do not say which debuggers	Book says to test it
Your VS Code symptom	I cannot see your setup	Ranked hypotheses (Appendix A)
Brooks and Kernighan exact wording; Beck’s “make the change easy”	Wording varies or the origin is unconfirmed	Paraphrased or not used

Item	What happened	Handling
Victor's "Inventing on Principle"	Video not readable	Secondary only; innovation claim marked an argument
Time budgets (chapter 20)	Not from research	Marked as a proposal
Gillian Minnehan's talk	Behind a login; I read only the session page	Your notes are the source for its contents

Credits

The thesis is yours: that the tool works only as well as the developer understands the problem. The sources above supply the evidence. The practice program, probes, and transcripts are in `lab/`.

APPENDIX E

Glossary

Plain-language meanings, as used in this book.

Term	Meaning
Address	Where a variable lives in memory. A pointer holds one.
Arm	Keep a probe switched off until another probe fires and turns it on.
Backtrace (`bt`)	The list of function calls that led to the current stop, innermost first.
Behavior Card	The one-page translation of a behavior into data and conditions (chapter 6).
Breakpoint	A *place* where the program stops.
Catchpoint	A breakpoint on an event class, such as a C++ exception or a signal.
Condition	A true-or-false expression that decides whether a breakpoint or watchpoint stops.
Confidence Gate	The explain, predict, locate test at the end of step 2.
Core dump	A file with the memory and thread state of a crashed process. You inspect it; you cannot run it.
Counterfactual	Changing a value by hand in the debugger to test a hypothesis before changing code.
Declare	What exists, and where: the data and places involved.
Define	What must be true, and when: initial values, expected values, the condition that makes the behavior right or wrong.
Defect	The line of code that is wrong. (Zeller.)
Dprintf	A gdb breakpoint that prints and continues.
Failure	What you can see go wrong. (Zeller.) Usually far downstream of the cause.
Family (A, B, C)	The kind of task: Understand, Fix, or Build.
Frame	One function call on the stack, with its own arguments and local variables.
Gate	A pass or fail check between steps.

Term	Meaning
Golden log	Saved probe output from before a change, used to `diff` after it.
Hit count	How many times a breakpoint location was reached.
Hypothesis	A testable statement about the cause. It makes a prediction.
Ignore count	Skip the next N times a breakpoint is reached.
Implement	What changes the data, and how: writers, readers, and the branches that decide.
Infection	The first bad program state. (Zeller.)
Instance	One object of a class, at one address. `this` points at it.
Invariant	A condition that must always hold.
Location watchpoint (`-l`)	A watchpoint on a memory location, not on a variable's scope.
Probe	A small instruction to the debugger: when this happens, do that. FILTER, COUNT, LOG, TRAP, ARM, REWIND.
Postcondition	What must hold after code runs (`Q` in `{P} C {Q}`).
Precondition	What must hold before code runs (`P`).
Slice	The part of the system on your card: everything that reads or writes your data.
State	The values of the data at a moment.
Stuck clock	The rule: a step over 1.5 times its budget triggers a reset and a question to someone.
Symbols	The names and line numbers a debug build carries so the debugger can show them.
TRAP	A probe that stops when data changes (a watchpoint).
Watchpoint	A *data* breakpoint: stops when a value is written (or read).
Writer / reader	Code that changes data / code that looks at it.